

SABANCI UNIVERSITY
Graduate School of Engineering and Natural Sciences
Information Technology Professional Master's Program

FREE CLOUD INITIATIVE: DESIGN AND IMPLEMENTATION OF A SELF-HOSTED CLOUD PLATFORM
ON BARE-METAL KUBERNETES

A Non-Thesis Master's Graduation Project Report Submitted in Partial Fulfillment of the
Requirements for the Degree of Master of Science in Information Technology

Prepared by:
ÖMER FARUK KARATAŞ (Student ID: 38160)

Declaration of Academic Integrity

I hereby declare that this graduation project report is my own original work conducted. All external sources, code repositories, frameworks, libraries, APIs, and literature used in this project have been appropriately cited and acknowledged in accordance with academic and professional ethics standards.

I also confirm that this work has not been submitted in whole or in part for any other degree or qualification at this or any other institution.

Student Name & Surname: Ömer faruk Karataş

Signature: 

Date: 02 / 09 / 2026

Abstract

Commercial cloud providers such as AWS, Google Cloud, Microsoft Azure, DigitalOcean, Linode, and Civo make infrastructure programmable, portable, and scalable, while also imposing recurring costs, provider-specific dependencies, and third-party control over data and execution environments. This project introduces the Free Cloud Initiative (FCI), a fully open-source platform that builds upon the core product model of these services, breaking them down into essential cloud elements suitable for community use, experimentation, and education.

FCI was implemented on a physical cluster of seven ARM64 Raspberry Pi 5 single-board computers. The limited memory, storage capacity, and virtualization capabilities of this hardware make a traditional virtual machine service very difficult. Therefore, container-based computing engines were designed as the primary computing abstraction, rather than being treated as a temporary substitute. The resulting Go microservice control plane exposes computation-wide computing [1], CloudNativePG-managed PostgreSQL [10, 11], Garage [12]-powered S3-compliant object storage, virtual networks and firewall policies mirrored in Kubernetes NetworkPolicy [5], browser-based WebSocket terminals, and centralized IAM. Public DNS management and secure edge routing are automated with Terraform via Cloudflare Zero Trust tunnels [9, 21], while non-production testing environments and CI runner instances are declaratively provisioned across external cloud providers using multi-cloud Terraform modules. Authentication provides OIDC authentication, while the API gateway generates ephemeral, audience-dependent Ed25519 JWTs. PostgreSQL stores persistent request state and reconciliation queues, and asynchronous workers aggregate Kubernetes and dataplane resources.

The platform is also intended to function as an end-to-end DevOps and platform engineering demonstrator. It launches Ansible, K3s, and OpenBAO [6, 8, 14]; implements Argo CD and Helm for GitOps reconciliation [7]; provides GitHub Actions for reusable test and image workflows; and offers monitoring for Prometheus [22], Grafana [23], Loki [25], Tempo [26], Alloy, and OpenTelemetry [24]. Cleaned Grafana views allow users and visitors to examine cluster health, making operational behavior a part of the learning experience rather than a hidden administrative detail.

FCI is free to use and designed to contribute to the open-source community. Its sustainability model relies on Patreon and community sponsorship, with all sponsorship revenue reinvested in additional physical servers and cluster capacity.

Keywords: Open-source cloud; bare-metal Kubernetes; DevOps; K3s; ARM64 edge computing; GitOps; cloud control plane; platform engineering

Table of Contents

1. Introduction
 - 1.1 Problem Statement & Background
 - 1.2 Project Objectives & Scope
 - 1.3 Stakeholder Analysis & Target Users
 - 1.4 Report Organization
2. Literature Review & Technology Stack Selection
 - 2.1 Related Work & Existing Systems
 - 2.2 Technology Stack Evaluation & Trade-off Analysis
3. Requirements Engineering & Specifications
 - 3.1 Functional Requirements
 - 3.2 Non-Functional Requirements
 - 3.3 System Constraints & Assumptions
4. System Architecture & Detailed Design
 - 4.1 High-Level Architecture
 - 4.2 Control-Plane and Reconciliation Architecture
 - 4.3 Data Architecture & Schema Design
 - 4.4 API & Interface Design
 - 4.5 Security, Authentication & Access Control
5. DevOps, Infrastructure & Delivery Pipeline
 - 5.1 Bare-Metal Bootstrap & Ansible Automation
 - 5.2 Containerization & Multi-Arch Build Strategy
 - 5.3 Continuous Delivery Workflows (GitHub Actions)
 - 5.4 Declarative GitOps: Argo CD App-of-Apps & Sync Waves
 - 5.5 Infrastructure-as-Code & Edge Automation
 - 5.6 Quality Gates, Linters & Security Tooling
6. Software Implementation & Microservice Architecture
 - 6.1 Shared Runtime (platform-common)
 - 6.2 Edge Ingress & API Gateway (api-gateway)
 - 6.3 Container-Native Compute Engine (compute-service)
 - 6.4 Managed PostgreSQL Engine (database-service)
 - 6.5 Object Storage & Virtual Networking (storage-service)
 - 6.6 Interactive WebSocket Terminal Gateway (terminal-gateway)
 - 6.7 Identity, Access & Audit Subsystem (iam-service)
 - 6.8 Frontend Web Application (frontend)
7. Verification, Testing & Evaluation
 - 7.1 Testing Strategy & Test Suites
 - 7.2 Requirements Traceability Matrix
8. Discussion, Limitations & Future Work

8.1 Post-Mortem & Objectives Realization

8.2 Technical Limitations & Known Issues

8.3 Future Roadmap & Extensions

9. Conclusion

10. References

List of Abbreviations

API: Application Programming Interface

ARM64: 64-bit Arm processor architecture, also known as AArch64

CI/CD: Continuous Integration and Continuous Delivery/Deployment

CNPG: CloudNativePG, a Kubernetes operator for PostgreSQL

CRD: Custom Resource Definition

FCI: Free Cloud Initiative

GitOps: A Git-centered model for declarative infrastructure and deployment operations

HA: High Availability

IAM: Identity and Access Management

JWT: JSON Web Token

K3s: A lightweight, conformant Kubernetes distribution

OIDC: OpenID Connect

RBAC: Role-Based Access Control

REST: Representational State Transfer

S3: Simple Storage Service-compatible object-storage API

SCIM: System for Cross-domain Identity Management

SPA: Single-Page Application

TLS: Transport Layer Security

VPC: Virtual Private Cloud

WebSocket: A full-duplex communication protocol operating over a persistent connection

1. Introduction

1.1 Problem Statement & Background

Offering infrastructure as a public cloud service transformed computing, databases, storage, network management, authentication, authorization, and operational tools into software engineering by making them available through programmable interfaces. A user creates an account, selects a resource, provides it via a control panel or API, and delegates infrastructure operations to the provider. The Free Cloud Initiative (FCI) draws its core product inspiration from leading providers such as Amazon Web Services (AWS), Google Cloud Platform (GCP), DigitalOcean, and Civo.

FCI is not seeking to replicate the extent, inventory diversity, territorial structure, or service assurance standards of a commercial supplier. The central inquiry is more specific: is it feasible to design and consistently deliver the essential experience of a public cloud on severely resource-limited bare-metal hardware, particularly a cluster of seven Raspberry Pi ARM64 nodes?

The fundamental resource collection comprises container computing engines, controlled PostgreSQL databases, S3-compatible object storage, account-specific virtual networks with firewall regulations, browser terminal access, IAM, quotas, and audit capabilities. Concentrating on these primitives enables the platform to address fundamental control-plane issues, including identity propagation, persistent desired state, reconciliation, isolation, secrets, observability, and deployment automation, while guaranteeing that the low-resource infrastructure can support multi-tenant workloads without succumbing to operational contention.

Operational transparency and real-time observability are equally crucial to this objective. Conventional cloud providers conceal physical infrastructure and control-plane internals under impenetrable service abstractions. Conversely, FCI is engineered to render cluster health, node usage, service lifecycle alignment, and physical capacity constraints explicitly observable and apparent. By including verified Grafana dashboards that users can access via their platform accounts, individuals may examine their allocated workloads as well as the real-time operating status, resource constraints, and health indicators of the foundational physical cluster.

Self-hosting presents an opposing issue: traditional private-cloud infrastructures are often tailored for enterprise-grade servers, exclusive network architectures, hardware virtualization, and specialist operational teams. FCI was established on a tangible cluster comprising seven Raspberry Pi ARM64 single-board computers. These devices offer an attainable, energy-efficient experimentation platform, although they enforce a rigid capacity limit. Cognitive capacity is finite, storage input/output and write longevity necessitate caution, pictures must be compatible with ARM64 architecture, and heat and power circumstances are operationally significant. Primarily, the cluster is incapable of supplying the necessary hardware and nested-virtualization framework essential for an effective general-purpose virtual-machine cloud. Simulating x86 virtual machines through software would deplete resources designated for productive tasks and yield deceptive performance metrics.

This hardware limitation directly influenced the computational abstraction. Rather than emulating a VM API on inadequate hardware, FCI conceptualizes a compute engine as a container workload scoped to an

account, incorporating defined CPU, memory, storage, lifecycle, network, and terminal functionalities. Kubernetes Deployments and persistent volumes serve as the execution foundation, but optional Kata support offers a further isolation profile contingent upon the compatibility of runtime and node capabilities. Container-native computing is therefore a purposeful technical solution to the seven-node Raspberry Pi ecosystem, rather than just a basic implementation expedient.

The initiative possesses an extra educational motivation. Effectively managing a beneficial cloud service entails more than merely launching application code. FCI integrates node preparation, K3s clustering, Ansible automation, OpenBao secret bootstrapping, Argo CD GitOps, Helm packaging, CI/CD, service-to-service identification, database migrations, asynchronous reconciliation, Kubernetes policy, and the Prometheus/Grafana/Loki/Tempo observability suite. The platform is designed to serve as an interactive environment where developers and infrastructure aficionados may allocate resources and see the operations of a cloud control plane. Sanitized Grafana dashboards are designed to display cluster health, capacity, service performance, and reconciliation to users and visitors while safeguarding credentials and tenant-sensitive information.

FCI therefore tackles an integrated engineering and societal issue. It seeks to determine if a cohesive collection of commercial cloud functionalities can be deployed on seven proprietary ARM64 nodes, presented via a reliable user-oriented platform, managed transparently using open DevOps methodologies, and augmented through an open-source funding framework. The desired outcome is not a seamless substitute for a hyperscale provider. It is a nimble cloud control plane that maintains authenticated, account-specific, API-based provisioning while rendering its physical constraints and operational processes transparent.

1.2 Project Objectives & Scope

The main aim of this graduation project is to conceptualize, execute, launch, and assess the Free Cloud Initiative as a replicable, entirely open-source substitute for the fundamental experience of commercial public clouds. The execution is anticipated to reveal valuable cloud resources via a unified web interface and HTTP API, while maintaining tenant ownership, persistent control-plane state, declarative operations, and interoperability with the seven-node ARM64 Raspberry Pi ecosystem. A concurrent aim is to provide a public educational platform whose structural framework may be analyzed, replicated, and enhanced by the community.

The project has the following concrete deliverables:

- Unified access layer: Develop a React [2]/TypeScript [3] application alongside an API gateway that offers a unified external interface for product APIs and web terminal sessions. The gateway is required to authenticate Authentik OIDC tokens and platform API keys, identify the associated account, implement edge controls, and substitute external credentials with ephemeral internal service tokens.
- Identity and access management: Implement accounts, IAM users, managed roles, policy evaluation, quotas, API-key lifecycle, Authentik synchronization, and append-oriented audit records. First login must provision a usable account and owner atomically.

- Container-native compute lifecycle management: Facilitate account-specific functionalities such as creation, inspection, initiation, cessation, rebooting, deletion, metrics, and backup information for computing engines intentionally designed as containers, as the Raspberry Pi cluster is incapable of sustaining a viable general-purpose VM service. The intended configuration must be integrated into Kubernetes Deployments, Services, persistent volumes, namespace quotas, and RBAC resources. The optional isolation of Kata Containers may be shown as a specific instance type alone when suitable nodes and runtime support exist; it does not transform the platform into a VM-centric cloud.
- Managed PostgreSQL: Facilitate the deployment of isolated PostgreSQL databases using the CloudNativePG operator, enable lifecycle and monitoring functionalities, accommodate constrained SQL execution and transactional CSV/JSON imports, and get credentials from Kubernetes Secrets without duplicating client passwords into the platform database.
- Interactive browser terminal: Establish a WebSocket-to-Kubernetes-exec gateway utilizing ephemeral, single-use Valkey tickets, ownership revalidation via the compute service, constrained per-account sessions, resizing notifications, idle time limits, and terminal activity monitoring.
- Reproducible platform operations: Utilize Ansible to initialize K3s and OpenBao, utilize Argo CD and Helm to synchronize infrastructure and apps from Git, implement External Secrets [17] to generate deployment credentials, and leverage GitHub Actions to verify, test, construct, and disseminate assets.
- Operational visibility and verification: Observe health probes, Prometheus metrics, organized logs, traces, and request correlation throughout backend services. Deliver unit, integration, contract, Kubernetes environment, frontend, Helm, and infrastructure validation appropriate for further requirements traceability. Disseminate purified Grafana dashboards that allow users and visitors to examine cluster health and capacity without obtaining passwords or sensitive tenant data.

The following boundaries are explicitly outside the project scope:

- A mobile application designed for native use; the compatible user interface is the responsive web single-page application and HTTP API.
- A multi-cluster control plane that operates at hyperscale or has geographical active-active capabilities. Terraform trials encompass several suppliers, although the executed product control plane is directed toward a singular K3s cluster.
- Infrastructure for general-purpose hypervisors akin to OpenStack Nova, Proxmox VE, or Harvester. Complete virtual machines are impractical on the project's Raspberry Pi infrastructure; FCI computing is deliberately designed for containerization, and Kata serves as an optional isolation tool rather than a virtual machine orchestration layer.
- Windows virtual machines, x86 emulation on ARM64 architecture, live migration capabilities, GPU resource allocation, and customizable virtual machine images.
- Billing, invoicing, payment facilitation, compensated resource levels, and marketplace features. Resource quotas and voluntary community sponsorship fall within the scope; however, monetary chargeback and preferential capacity allocation to sponsors do not.
- Oversight database systems apart from PostgreSQL.

- Comprehensive software-defined networking with virtual routers, overlapping tenant subnets, NAT gateways, VLAN orchestration, or an accurate depiction of deny and ICMP firewall semantics.
- Route customer credentials to the foundational Garage deployment or publicly presigned object URLs. Access to objects is facilitated by the storage service.
- A compute-restore API designed for customer interaction. Restoration is intentionally confined to an operator's process, incorporating supplementary safety measures.

They differentiate a community-supported free platform from a commercial hosting service: while sponsorship may enhance the collective physical resource pool, it does not modify IAM permissions, quotas, or technical acceptance standards.

1.3 Stakeholder Analysis & Target Users

FCI caters to several stakeholder groups with overlapping demands, although their objectives on risk, usability, education, and sustainability diverge.

- Independent developers and self-hosters: They necessitate an uncomplicated method to access cloud-like computing, database, storage, networking, identity, and terminal resources without the need to manually create Kubernetes manifests for each task. They engage predominantly with the frontend, API gateway, computational service, storage service, and browser terminal. Their goals include no usage costs within the accessible community capacity, clear management, transparent restrictions, and the capability to replicate the platform on their own hardware.
- Small engineering teams: They necessitate a unified platform for development environments, internal tools, and presentations. Their goals encompass account-specific access, reliable quotas, automation of API keys, audit capabilities, and role-based access management for shared resources.
- Students, DevOps learners, and platform-engineering enthusiasts: They utilize FCI as a dynamic technological laboratory. Their aim extends beyond mere resource consumption; they also get insights from monitoring the progression of a request through authentication, persistent state, reconciliation, Kubernetes scheduling, storage, policy, and telemetry. Publicly accessible, sanitized Grafana dashboards facilitate this function by revealing cluster health, capacity, and service performance while preserving security parameters. The repository collection offers a tangible comprehensive illustration including Terraform, Ansible, K3s, OpenBao, Argo CD, Helm, GitHub Actions, microservices, and observability.
- Open-source contributors: The group comprises developers, security evaluators, documentation writers, infrastructure managers, and researchers who enhance the platform via public cooperation. They necessitate modular repositories, defined service ownership, replicable development environments, reliable cross-service agreements, automated testing, and a strong differentiation between existing functionality, optional features, and experimental multi-cloud initiatives. The open-source methodology allows for autonomous deployments and forks instead than requiring participation to be contingent on the hosted FCI instance.
- Community sponsors and patrons: They offer discretionary monetary assistance for communal infrastructure. They are engaged in transparency and the accountable allocation of resources, although sponsorship does not provide administrative power, elevated IAM positions, or special

access for tenants. The declared sustainability policy designates all sponsorship revenue for supplementary physical servers and related capacity. This renders cluster expansion visible and aligned with the purpose, while circumventing an invoicing subsystem within the product.

This stakeholder framework fosters a clear yet secure architecture: users and teams engage through the web dashboard and API, learners and the community obtain operational visibility through open repositories and refined Grafana views, while low-level cluster credentials are rigorously segregated from tenant access.

1.4 Report Organization

Chapter 1 delineates the issue, aims, extent, participants, and rationale for the Free Cloud Initiative. Chapter 2 examines notable hyperconverged infrastructure and Kubernetes PaaS frameworks while elucidating the rationale behind the choice of the FCI technological stack. Chapter 3 delineates functional specifications, quantifiable quality criteria, limitations, and operational premises. Chapter 4 delineates the system architecture, the control-plane reconciliation framework, data structuring, interfaces, and security demarcations. Chapter 5 explores the bare-metal bootstrap, container supply chain, continuous delivery workflows, declarative GitOps operations, infrastructure as code, and automated quality gates. Chapter 6 delineates the software execution of the shared runtime, gateway, backend microservices, reconciliation algorithms, data-plane connections, and frontend. Chapter 7 assesses the system using automated testing, Kubernetes integration tasks, performance evaluations, and a requirements traceability matrix. Chapter 8 rigorously evaluates accomplished goals, recognized constraints, implementation deficiencies, and prospective advancements. Chapter 9 encapsulates the engineering contributions and conclusions of the project. Chapter 10 enumerates the formal technical references employed in the study, while the appendices furnish details on repository, interface, setup, and verification.

2. Literature Review & Technology Stack Selection

2.1 Related Work & Existing Systems

FCI is situated in relation to two recognized methods of providing cloud-like infrastructure. The initial model is the commercial public-cloud framework exemplified by hyperscale providers like Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure, alongside developer-centric providers such as Linode, DigitalOcean, and Civo. The second kind is self-regulated infrastructure utilizing open-source platforms like OpenStack and Proxmox Virtual Environment (VE). These systems create significant functional and operational benchmarks; yet, their foundational resource assumptions are fundamentally distinct from the seven-node Raspberry Pi framework for which FCI was developed.

Hyperscale and Developer Public Clouds

AWS, GCP, and Microsoft Azure set the modern standard for self-service infrastructure. Users are able to establish identities, assign computing resources, supply managed databases, generate object-storage

namespaces, configure virtual networks and firewall regulations, examine telemetry, and automate these processes using APIs and infrastructure-as-code instruments. Their control planes obscure the aspects of placement, failover, hardware preservation, capacity management, and a significant portion of the service lifecycle through reliable resource abstractions. Developer-centric platforms like Linode, DigitalOcean, and Civo employ a comparable framework including streamlined catalogs and interfaces designed to alleviate the initial operational load for individuals and small teams.

This product model has a direct impact on FCI. The compute engines are analogous to public-cloud compute instances; CNPG-managed PostgreSQL functions as a restricted managed-database service; Garage-supported logical buckets offer an S3-compatible storage abstraction; network and firewall records signify a purposely restricted VPC layer; the terminal gateway facilitates interactive workload access; and IAM administers accounts, users, roles, API keys, quotas, and activity logs. The React dashboard and HTTP API are both structured to facilitate self-service and account-specific resource management instead of providing direct access to cluster administration.

FCI does not replicate the proprietary execution or complete semantics of any specified provider. Public-cloud control planes are proprietary operational frameworks implemented within extensive regional data centers, tailored networks, dedicated storage arrays, and significant automation. Their financial framework include ongoing fees for designated resources, managed services, requests, transfers, and assistance. Their service-oriented identity, policy, networking, monitoring, and deployment capabilities may potentially induce technical and operational entrapment. FCI embraces fundamental user experience and primary resource categories, substituting provider-managed elements with open-source Kubernetes workloads and operators operating on proprietary hardware. The resultant system is verifiable and replicable; yet, it does not assert the flexibility, geographical redundancy, hardware variety, or service-level assurances typical of a commercial provider.

Should corporate data-center resources be accessible, a configuration utilizing hardware-virtualized instances, exclusive storage and network layers, redundant control nodes, and provider-scale schedulers would be technically feasible. That option would more accurately reflect the architecture supporting AWS, GCP, Azure, or Linode. The feasibility of FCI was compromised due to the presence of seven ARM64 Raspberry Pi single-board computers, which possess restricted memory, I/O throughput, and virtualization capabilities. The pertinent design aim was therefore not to condense a full hyperscale architecture, but to maintain its most beneficial product interactions within a much reduced resource allocation.

OpenStack as an Open-Source Private Cloud

OpenStack serves as the primary open-source standard for an all-encompassing private infrastructure-as-a-service solution. The service architecture comprises Nova for computing, Neutron for networking, Keystone for identity management, Glance for image storage, Cinder for block storage, Swift for object storage, and Horizon for dashboard interface. The architecture is modular and reliant on APIs: distinct services typically segregate API functions from workers, maintain state in relational databases, and manage asynchronous tasks using an AMQP-compatible message broker. This framework

accommodates various hypervisors, tenant networks, image repositories, storage drivers, availability zones, scheduling protocols, and extensive administrative implementations.

OpenStack would serve as a suitable platform if FCI has server-class computing nodes, hardware virtualization, dedicated storage, redundant networking, and adequate operating capacity. It effectively addresses several issues associated with private clouds that FCI examines and provides significantly more extensive infrastructure capabilities than what a graduation project should endeavor to replicate. Opting for OpenStack in such circumstances might be more justifiable than developing an alternative general-purpose IaaS control plane.

The intended hardware contradicts that assertion. A purposely constrained OpenStack deployment, nonetheless, initiates several enduring APIs and workers, service-specific persistent states, message-broker accessibility, network agents, image administration, and hypervisor integration prior to the utilization of resources by user workloads. The control-plane footprint and related failure domains on seven Raspberry Pi nodes would directly contend with the workloads the platform is designed to accommodate. Complete hardware virtualization and nested hypervisors are not a feasible execution paradigm within the cluster, whereas software emulation would incur intolerable CPU and memory expenses.

FCI, therefore, preserves many OpenStack ideas without embracing its deployment framework. It maintains separately operated services, account-specific APIs, persistent state, asynchronous resource generation, and clear identity demarcations. It substitutes the conventional hypervisor scheduler with K3s, the AMQP-centric work graph with PostgreSQL-supported reconciliation queues, and the dedicated infrastructure agents with Kubernetes and operator control loops. CNPG constructs PostgreSQL clusters, Garage offers object storage solutions, and NetworkPolicy delivers a securely expressible portion of tenant firewall functionality. This decrease does not imply that OpenStack is inherently inefficient; rather, it is an architectural adjustment to a fundamentally altered capacity framework.

Proxmox Virtual Environment

Proxmox VE is a Debian-based virtualization platform that is open-source and incorporates KVM virtual machines alongside LXC containers. It offers a sophisticated web interface and API for managing hosts and guests, clustering, storage, backup, migration, high-availability oversight, and firewall setup. Its integrated KVM/LXC framework renders it ideal for laboratories, domestic setups, and private settings where server hardware offers virtualization enhancements and the primary objective is to manage virtual instances effectively (Proxmox Server Solutions, 2025).

Proxmox is therefore a valid choice for a distinct FCI hardware situation. Utilizing x86-64 or adequately proficient ARM servers, it may accommodate complete operating-system virtual machines and offer superior guest compatibility, isolation, migration, and hardware-passthrough capabilities compared to FCI's container engines. It possesses a much reduced conceptual footprint compared to OpenStack for administrators requiring virtualization management instead of a whole cloud-service portfolio.

The difference lies not in Proxmox's absence of automation or access control; rather, it provides both. The differentiation lies in the alignment of its control plane. Proxmox focuses on administrators

overseeing hosts, KVM virtual machines, LXC containers, storage solutions, and cluster assets. FCI focuses on users administering account-specific cloud offerings via a consolidated application API. PostgreSQL management, S3-compatible logical buckets, product IAM and API keys, quotas, reconciliation status, browser terminal tickets, service-level telemetry, and Argo CD-managed platform deployment are incorporated into FCI as product functionalities instead of being constructed by a Proxmox administrator for separate guests.

The seven Raspberry Pi nodes undermine the principal benefit of a KVM-centric architecture: effective full-system virtualization. LXC may facilitate container execution; however, utilizing Proxmox just for containers would maintain a host-virtualization management framework while lacking the Kubernetes operators, declarative workload APIs, and GitOps integration essential for the initiative. K3s is hence the most suitable foundation for the chosen resources and educational goals.

Resource-Constrained Architectural Position of FCI

The primary engineering inquiry is: How can the fundamental public-cloud user experience be provided when the existing infrastructure is inadequate for a traditional virtual-machine cloud or a robust private IaaS control plane?

FCI addresses this inquiry by considering containers and Kubernetes reconciliation as fundamental architecture elements. K3s orchestrates computation engines directly on ARM64 nodes. PostgreSQL retains the preferred product condition and persistent work assertions. Go services execute account-centric APIs, establish security demarcations, enforce quotas, and facilitate resource-specific reconciliation. Current open-source frameworks execute certain data-plane functions: CloudNativePG oversees PostgreSQL clusters, Garage retains objects, Kubernetes NetworkPolicy implements the expressible aspect of firewall intentions, Authentik delivers OIDC identity, and OpenBao offers secrets. Argo CD perpetually aligns the platform directly from Git.

This method permits clear limitations in scope. FCI does not support random guest operating systems, live virtual machine migration, extensive provider-scale regions, comprehensive software-defined networking, or hyperscale flexibility. In exchange, it provides the chosen cloud primitives within the CPU, memory, storage, and virtualization constraints of seven Raspberry Pi nodes, while ensuring the control plane remains accessible, transparent, and replicable. Its contribution is hence not a mere replica of AWS or OpenStack; it is an evidence-driven reconfiguration of the cloud interaction paradigm for an extreme edge and educational resource framework.

2.2 Technology Stack Evaluation & Trade-off Analysis

The selection of technology was dictated by five factors: the availability of ARM64, runtime footprint, compliance with declarative Kubernetes operations, maintainability by a compact team, and the capacity to substitute proprietary managed services with open interfaces. The chosen stack and its key trade-offs are presented below.

Component Layer	Selected Technology	Alternatives Considered	Architectural Rationale and Accepted Trade-off
Cluster orchestration	K3s on ARM64	Upstream Kubeadm, MicroK8s, Nomad	K3s is a compliant Kubernetes distribution that offers a minimal operating footprint and certified support for ARM64 architecture. It consolidates containerd, Flannel, CoreDNS, kube-router policy capabilities, local-path provisioning, and an integrated registry mirror. FCI deactivates the bundled Traefik and ServiceLB to ensure that GitOps-governed Traefik and MetalLB have clear ownership. The compromise is reliance on K3s networking characteristics, particularly the existing restrictions on egress policies for service ClusterIP traffic.
Backend services	Go 1.26.6 with "net/http"	Node.js/NestJS, Java/Spring, Rust/Axum	Go generates statically linked ARM64 executables, exhibits a very minimal memory use, and offers direct concurrency and cancelation mechanisms for HTTP servers, workers, and Kubernetes clients. A communal module standardizes issues without the need for a cumbersome service structure.
Durable control-plane database	PostgreSQL 16 operated by CloudNativePG	MySQL, SQLite, CockroachDB, externally managed PostgreSQL	PostgreSQL offers transactions, row-level locking, advisory locks, JSON functionality, and advanced indexing for account-specific states and persistent queues. CNPG offers declarative administration of clusters and the creation of secrets using Kubernetes. The platform employs distinct schemas and roles for the management of services.
Managed customer databases	PostgreSQL through CloudNativePG CRDs	Direct StatefulSets, external managed databases, multiple database engines	Utilizing CNPG enables FCI to execute the database lifecycle via reconciliation instead of bespoke replication and failover mechanisms. Credentials are retained within Secrets controlled by CNPG. Limiting the first service to PostgreSQL narrows its scope but enhances the testability of lifecycle, SQL execution, import, and isolation.
Object storage	Garage with an S3-compatible API	MinIO, Ceph Object Gateway, filesystem storage	Garage offers a lightweight, self-hosted storage solution compatible with S3. FCI utilizes account prefixes to represent a singular physical bucket, hence preventing the disclosure of backend credentials. The compromise is that tenant access regulations are implemented by the platform proxy instead of being natively enforced by Garage.
Identity provider	Authentik using OIDC	Keycloak, Dex, Zitadel, custom authentication	Authentik oversees authentication and OIDC federation, whereas internal IAM governs authorization, quotas, and API keys. The compromise involves eventual consistency, as IAM modifications synchronize with Authentik asynchronously through its administrative API.
Secrets management	OpenBao with External Secrets	HashiCorp Vault, Sealed Secrets, plaintext Kubernetes Secrets	OpenBao provides a Vault-compatible secret store with Kubernetes authentication. Ansible installs, unseals, and seeds it before Argo CD syncs, allowing External Secrets to inject scoped credentials into namespaces without storing bootstrap tokens in Kubernetes.

GitOps controller	Argo CD and Helm	Flux CD, Jenkins X, imperative "kubect!" deployment	Argo CD offers declarative application reconciliation, synchronization waves, drift remediation, pruning, and a hierarchical application model that encompasses both infrastructure and product charts.
CI/CD	GitHub Actions (self-hosted runners)	GitLab CI, Jenkins, local scripts	Organization-level workflows standardize Go race tests, frontend checks, Helm/Terraform/Ansible validation, and ARM64 GHCR builds across Terraform-provisioned runner fleets. This minimizes pipeline duplication while depending on GitHub availability and careful workflow versioning.
Frontend	React 19, TypeScript 6, Vite 8	Next.js, SvelteKit, server-rendered templates	A client-side single-page application matches interactive dashboard and terminal requirements, using runtime configuration, TanStack Query [27], and Zustand [28] to manage state without environment-specific rebuilds.
Edge ingress	Cloudflare Tunnel and Traefik	Public node ports, hardware load balancer, VPN-only access	Cloudflare Tunnel enables public ingress behind NAT without open inbound ports, while Traefik handles internal cluster routing. However, external access remains dependent on Cloudflare's control plane and tunnel availability.
Ephemeral coordination	Valkey	Redis, in-process caches, PostgreSQL-only coordination	Valkey [13] handles rate limits, idempotency, auth caching, terminal tickets, and session slots. Routine caching fails open, whereas ticket redemption fails closed. As an ephemeral store, it persists no durable business state.

K3s serves as the fundamental choice. The official K3s documentation recognizes edge systems, homelabs, and ARM single-board computers as primary applications and provides direct support for `arm64/aarch64`. This circumvents cross-architecture emulation and allows the service images and data tasks to be constructed for their inherent execution environment. K3s maintains the Kubernetes API, enabling FCI to utilize conventional Deployments, Services, Secrets, RBAC, ResourceQuota, NetworkPolicy, and operator CRDs rather than creating a custom scheduling interface.

The integration of Go with PostgreSQL illustrates the distinction between operational efficiency and state accuracy. Go services are consistently horizontally scalable and possess minimal persistent in-process state. PostgreSQL transactions safeguard account provisioning, last-administrator regulations, quotas, preferred resource records, and job entitlements. Workers engaged in reconciliation employ database locking strategies like `FOR UPDATE SKIP LOCKED` to allocate tasks across replicas without the necessity of an additional message broker. Valkey is designated for data that might become obsolete or be reconstituted.

The chosen data-plane elements are intentionally tailored. CNPG possesses the mechanics of the PostgreSQL cluster, Garage is responsible for object durability, Longhorn provides block storage when necessary, and Kubernetes manages workload scheduling and the implementation of network policies. FCI's offerings synchronize these systems and provide reliable account-centric APIs; they do not seek to replicate the underlying mechanisms of a database operator or object storage.

At the heart of the Free Cloud Initiative's concept is an unwavering dedication to open-source and publicly accessible software. Each runtime, database engine, operator, and infrastructure tool inside the design was chosen to eradicate proprietary licensing expenses and dependency on commercial vendors. By depending solely on open-source alternatives (like OpenBao in place of proprietary Vault, Garage for S3 object storage, Authentik for identity management, and CloudNativePG for PostgreSQL), the platform guarantees that its complete control plane and data plane are subject to unrestricted auditing, deployment, and reproduction by any self-hosting entity or educational institution without monetary constraints.

Ultimately, the delivery stack renders the cluster replicable without asserting that bootstrap is exclusively GitOps. Ansible governs the dependency-critical beginning stage: node configuration, K3s establishment, OpenBao installation and initialization, and Argo CD setup. Argo CD maintains ongoing reconciliation once the necessary conditions are fulfilled. This separation prevents a cyclic dependence where External Secrets would necessitate an operational secret store that has not been established. Traefik [20] handles internal cluster routing.

3. Requirements Engineering & Specifications

3.1 Functional Requirements

- FR-01 (Authentication, Account Resolution, and RBAC): The system will verify users with Authentik OIDC and automate processes using Argon2id-hashed API keys, generating ephemeral Ed25519 JWTs for internal service navigation. IAM will oversee accounts, quotas, and RBAC roles with an examination of deny overrides while guaranteeing the presence of at least one active administrator. Acceptance is confirmed by contractual agreements, role permission, border segregation, and secret safeguarding assessments.
- FR-02 (Compute Engine Lifecycle): Verified accounts will establish, oversee, and softly remove containerized computing instances over the HTTP API, which maintains state and synchronizes Kubernetes `Deployments`, `Services`, and volumes. Instance requests must adhere to account limitations, accommodate optional Kata isolation with a fail-closed mechanism, and limit recovery to actions performed by the operator. Validation of acceptance is conducted using lifecycle assessments, queue transaction evaluations, namespace segregation, and quota enforcement examinations.

3.2 Non-Functional Requirements

- NFR-01 (Observability and Diagnosability): All requests must transmit trace identifiers and organized log context throughout internal services, revealing Prometheus metrics, Loki logs, and Tempo traces. Standard health, readiness, and diagnostic metrics must reveal troubleshooting status while safeguarding sensitive information.
- NFR-02 (GitOps Portability and Reproducibility): All infrastructure and apps will be controlled declaratively in Git with Argo CD, incorporating automatic drift remediation. The platform should be

entirely replicable on ARM64 nodes utilizing Ansible and runtime setup, free from dependency on public cloud providers.

3.3 System Constraints & Assumptions

The requirements are interpreted under the following physical, architectural, and operational constraints.

- **ARM64 hardware specifications:** The main focus is on Raspberry Pi or similar ARM64 Linux devices that possess much lower CPU, memory, storage capacity, and temperature tolerance compared to data center servers. Consequently, all product visuals and data tasks must be accessible for ARM64. QEMU-based x86 emulation is omitted due to its overhead potentially compromising resource and performance assumptions.
- **Storage characteristics:** The efficacy of the Kubernetes control plane and database is heavily influenced by disk latency and write durability. SSD-based storage is presumed for write-heavy control-plane and database elements whenever feasible. Longhorn, local-path volumes, Garage replication, and CNPG replication safeguard many failure scenarios and should not be seen as replacements for an external backup plan.
- **Single-cluster topology:** It is presumed that the production control plane functions within a singular K3s cluster, where the nodes are interconnected via a reliable LAN and may access the Kubernetes API and one another using the necessary K3s networking ports. Multi-cloud Terraform repositories serve as infrastructure trials and provisioning tools; they do not render the application control plane multi-cluster.
- **Kubernetes ownership:** Kubernetes is regarded as the definitive runtime API for managing workloads, namespaces, RBAC, persistent-volume claims, CNPG resources, and NetworkPolicies. Operators should refrain from executing unmonitored live modifications to GitOps-managed entities and anticipate their permanence. Reconciliation is ultimately consistent, hence a successful creation answer indicates that the intended state has been acknowledged, rather than confirming that the resource is now operational.
- **Ingress dependency:** Public access is presumed to be facilitated using a Cloudflare Zero Trust tunnel and Traefik. The cluster needs outward network access to Cloudflare, appropriate DNS and tunnel setup, and an operational cloudflared deployment. A Cloudflare disruption might eliminate public accessibility despite the local system being operational.
- **Git and image availability:** Argo CD necessitates read permissions for the manifest repositories, while nodes must have access to the necessary GHCR images or to images that are already present via the K3s integrated Spegel mirror. The absence of Git does not instantaneously halt active resources, nevertheless it obstructs the synchronization of the new intended deployment state. The absence of the image source may hinder scheduling following cache eviction or node substitution.
- **Bootstrap trust:** The initial setup of OpenBao, including installation, startup, unsealing, secret seeding, and Argo CD bootstrap, need an authorized operator and external secret inputs. This trust ceremony cannot be assigned to External Secrets, as External Secrets relies on a properly configured OpenBao instance.

- The capacity is limited: Quotas inhibit new distributions above established account thresholds, although they do not instantly eliminate current resources when an operator reduces a quota. The physical cluster must maintain sufficient capacity for system components, reconciliation, backup operations, and failure recovery alongside customer demands.
- Container-native computation: Shared computing engines consist of Kubernetes containers equipped with persistent volumes instead of conventional virtual machines. Dedicated execution presupposes an operational Kata RuntimeClass and a qualifying labeled node. The existing Ansible Kata installation play is inactive, and Raspberry Pi-class nodes often lack the capability for effective layered virtualization, so dedicated computing should be regarded as conditional rather than universally accessible.
- Network-policy expressiveness: The Kubernetes NetworkPolicy framework is incapable of encapsulating the entirety of conventional firewall semantics. FCI posits that an allow-oriented TCP/UDP policy is beneficial for tenant segregation, although it does not assert complete VPC routing, equivalence in stateful firewall functionality, or enforceable deny/ICMP regulations. Furthermore, the K3s kube-router's functionality inside the existing topology hinders the secure limitation of some service egress routes, prompting certain application rules to deliberately permit uncontrolled egress.
- Object-storage facilitation: Access to the garage is presumed to be attainable just via trusted platform credentials and cluster networking. Access to customer objects is mediated by the storage service. Logical access-policy documentation indicates expected permissions, although Garage does not independently enforce them when utilizing a shared platform credential.
- External identity availability: It is presumed that Authentik will continue to be accessible for browser-based authentication and administrative synchronization. IAM continues to have jurisdiction over FCI accounts, roles, quotas, API keys, and auditing. A malfunction in post-commit Authentik synchronization should be apparent and resolvable, although it must not reverse a previously committed local transaction.
- Temporary Valkey state: Valkey operates without permanence and may forfeit rate-limit intervals, caches, idempotency logs, unutilized console tickets, and session-slot documentation upon restart. No sustainable company asset should rely exclusively on Valkey. Cache-dependent processes could fail in an open state while maintaining accuracy, although the redemption of terminal tickets must result in a closed failure.
- Operational management: It is presumed that a reliable operator possesses SSH and Kubernetes administrator privileges for initialization, incident management, certificate remediation, data recovery, and secure compute restoration. Tenant users are not provided with Kubernetes credentials, Garage credentials, or direct access to the PostgreSQL schemas of the platform.

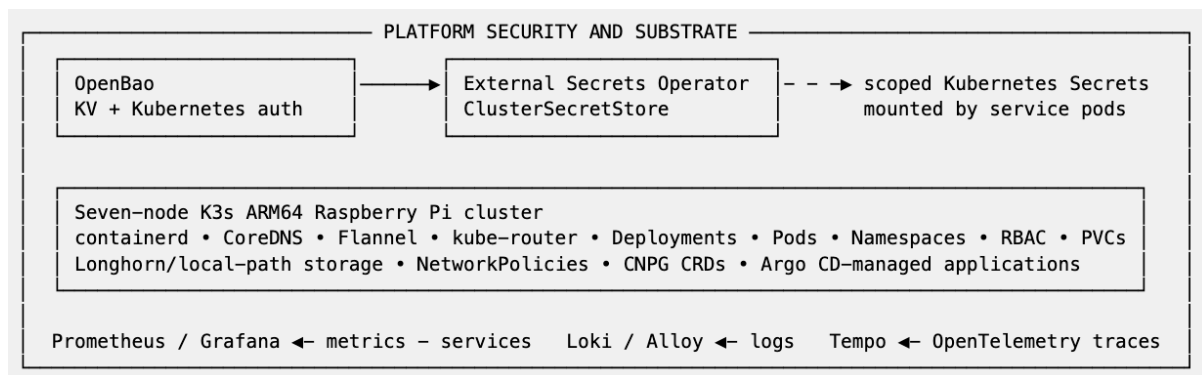
Under these premises, FCI may be assessed as a consolidated single-cluster cloud infrastructure. Assertions concerning hyperscale flexibility, geographical consistency, total virtual-machine segregation, or equivalence with public-cloud services are beyond the scope of the design's validity and should not be deduced from the execution.

4. System Architecture & Detailed Design

FCI is structured as a stratified cloud management framework. External clients engage with a consolidated edge that authenticates identity and directs requests to domain services, while asynchronous workers harmonize persistent intent via Kubernetes without revealing foundational credentials to tenants.

4.1 High-Level Architecture

Figure 4.1 illustrates the primary containers and external dependencies within a C4-style container representation. Solid arrows denote pathways for requests or data. Dashed arrows signify clandestine materialization, reconciliation, or observability routes instead of standard tenant requests.



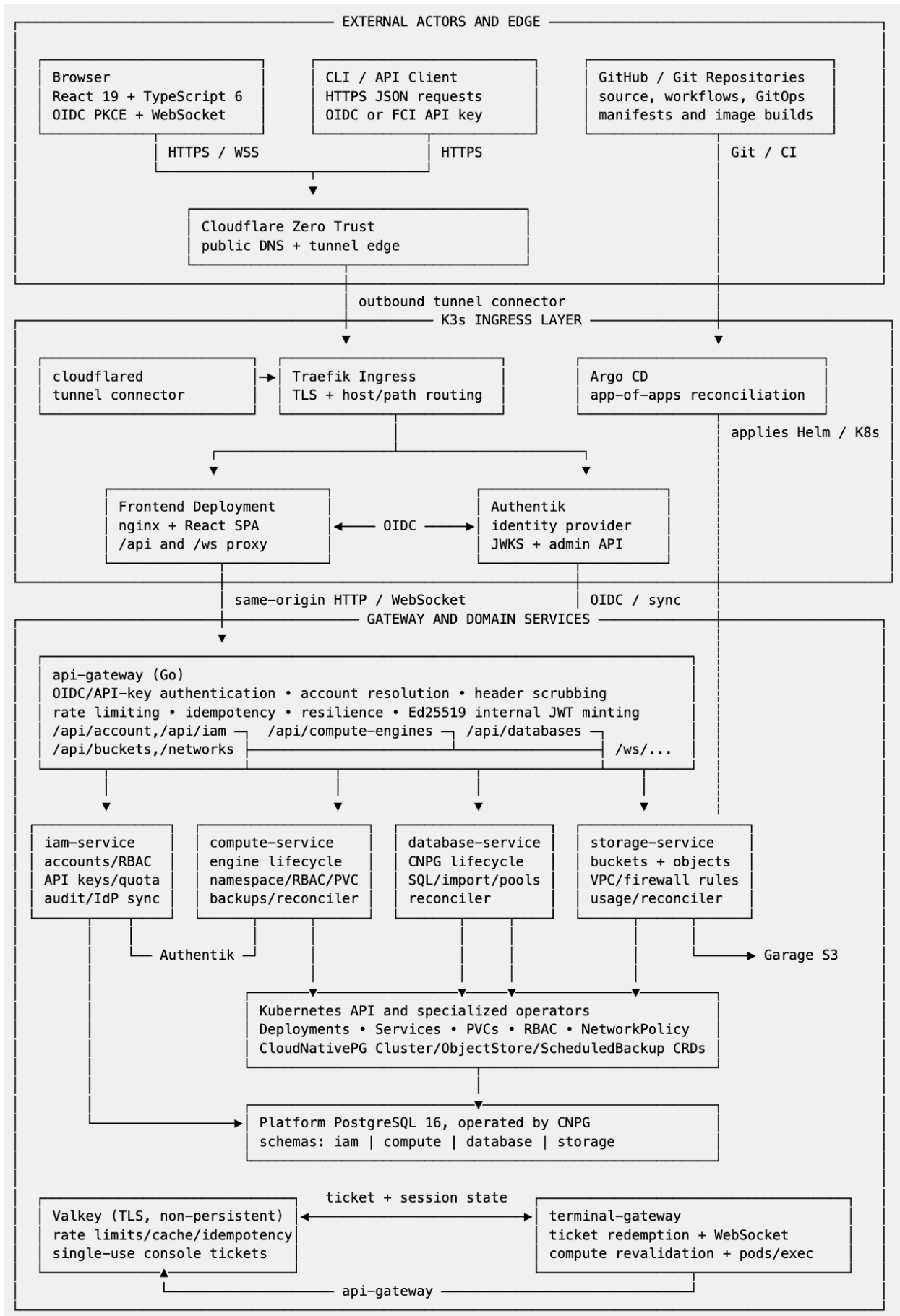


Figure 4.1: C4-style container view of the Free Cloud Initiative.

System Boundary and External Access

The public product boundary is deliberately narrower than the cluster boundary. Public DNS and Cloudflare Zero Trust direct traffic via an outward `cloudflared` connection to Traefik. Traefik serves as the ingress controller for the cluster and manages the external routing decisions. The primary application entry point focuses on the frontend `Deployment`. The `nginx` container delivers the built SPA and forwards both `/api/` and `/ws/` requests to `api-gateway`, maintaining a same-origin policy for the browser. As a result, the browser fails to identify or access the compute, database, storage, IAM, or terminal services via the Kubernetes service name.

Authentik constitutes an independent identification system accessible from external sources. The SPA executes Authorization Code with PKCE via Authentik, while the gateway authenticates the resultant OIDC access token against the designated issuer and JWKS. Automation customers could alternatively provide an FCI API key. Both credential categories align on the identical internal actor depiction prior to the domain service processing the request.

Gateway and Service Responsibilities

`api-gateway` is an edge security and routing component, not a business-data owner. Its responsibilities include credential dispatch, OIDC validation, API-key verification through IAM, first-login account resolution, identity-header removal, request identifiers, tracing, account-scoped rate limiting, optional mutation idempotency, upstream timeout and circuit-breaker behavior, and internal JWT minting. It owns the mapping from public route prefix to internal audience. Ordinary HTTP traffic is sent to IAM, compute, database, or storage; WebSocket upgrades are sent to `terminal-gateway` through a structurally separate handler so that long-lived connections do not inherit ordinary request timeouts or retry behavior.

The five domain processes have explicit ownership:

- `iam-service` owns the account boundary, users, managed roles, policies, quotas, API keys, audit history, and best-effort Authentik synchronization.
- `compute-service` owns the desired lifecycle of compute engines and reconciles Kubernetes namespaces, quotas, limits, RBAC, image-pull credentials, Deployments, Services, PVCs, metrics, and disk-backup jobs.
- `database-service` owns managed-database intent, projects CNPG resources, reads generated database credentials from Kubernetes Secrets, and provides bounded SQL, import, connection, and metrics operations.
- `storage-service` owns logical buckets, object mediation, usage snapshots, reserved backup buckets, virtual networks, firewall records, and NetworkPolicy projection.
- `terminal-gateway` owns no durable product database. It redeems short-lived tickets, revalidates the compute target, acquires a bounded session slot, and bridges a WebSocket to Kubernetes `Pods/exec`.

`platform-common` is not shown as a running container because it is a compile-time Go module. It supplies the internal JWT contract, actor context, HTTP middleware and error envelope, PostgreSQL and migration helpers, Valkey primitives, configuration loading, health checks, logging, OpenTelemetry integration, and test support used across services.

Synchronous and Asynchronous Boundaries

The synchronous request pathway executes tasks that must be finalized prior to a secure response being feasible: credential authentication, account identification, request verification, quota assessment, authorization, transaction finalization, and acquisition of pre-existing state. The process of provisioning is inherently asynchronous. Calculations, database alterations, and network modifications generate a persistent intended state and a reconciliation task, subsequently yielding a representation that may retain a `pending` status. Employees then interact with the Kubernetes API or an operator and document the observed condition. This inhibits sluggish scheduling, image retrieval, PVC binding, CNPG failover, or NetworkPolicy implementation from maintaining an edge HTTP connection.

Object transfer and SQL execution are anomalies as the initiator specifically demands a synchronous data transaction. They are constrained by physical dimensions, number of rows, byte limitations, duration of statements, overall duration, pool acquisition, and transfer restrictions. Terminal sessions utilize an enduring WebSocket connection with distinct settings for idleness, maximum duration, keepalive, and draining.

ARM64 Resource Allocation

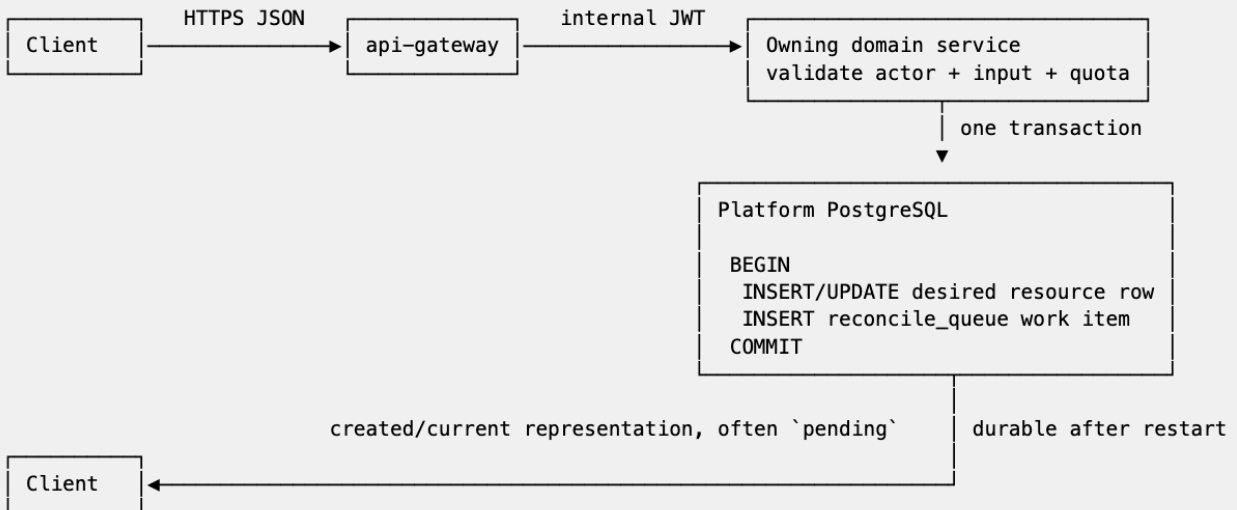
The seven-node Raspberry Pi cluster requires explicit resource budgeting. The frontend and all six Go processes are deployed with two replicas in the current Helm values. Most Go services request approximately 100 millicores and 128–256 MiB of memory; terminal-gateway requests 50 millicores and 64 MiB; upper limits prevent a single process from consuming a node. PodDisruptionBudgets retain at least one replica during voluntary disruption, and topology-spread rules attempt to place replicas on different eligible nodes without making a second replica permanently unschedulable when taints reduce placement options.

Stateful components use different durability strategies. Garage runs three replicas and performs application-level replication while each replica uses local Longhorn-backed storage. Valkey is a single TLS-protected, non-persistent instance because it contains only reconstructable or expiring coordination state. Platform PostgreSQL 16 is operated by CNPG as a single instance utilizing local-path storage. Customer databases are represented by CNPG resources whose requested sizes are subject to account quotas and cluster capacity. Low-memory nodes may be tainted, while node labels and tolerations constrain storage, control, or dedicated-runtime placement.

4.2 Control-Plane and Reconciliation Architecture

The central design pattern is durable desired state followed by asynchronous convergence. Figure 4.2 shows the common lifecycle used by compute engines, managed databases, and network/firewall resources.

SYNCHRONOUS ACCEPTANCE PATH



ASYNCHRONOUS CONVERGENCE PATH

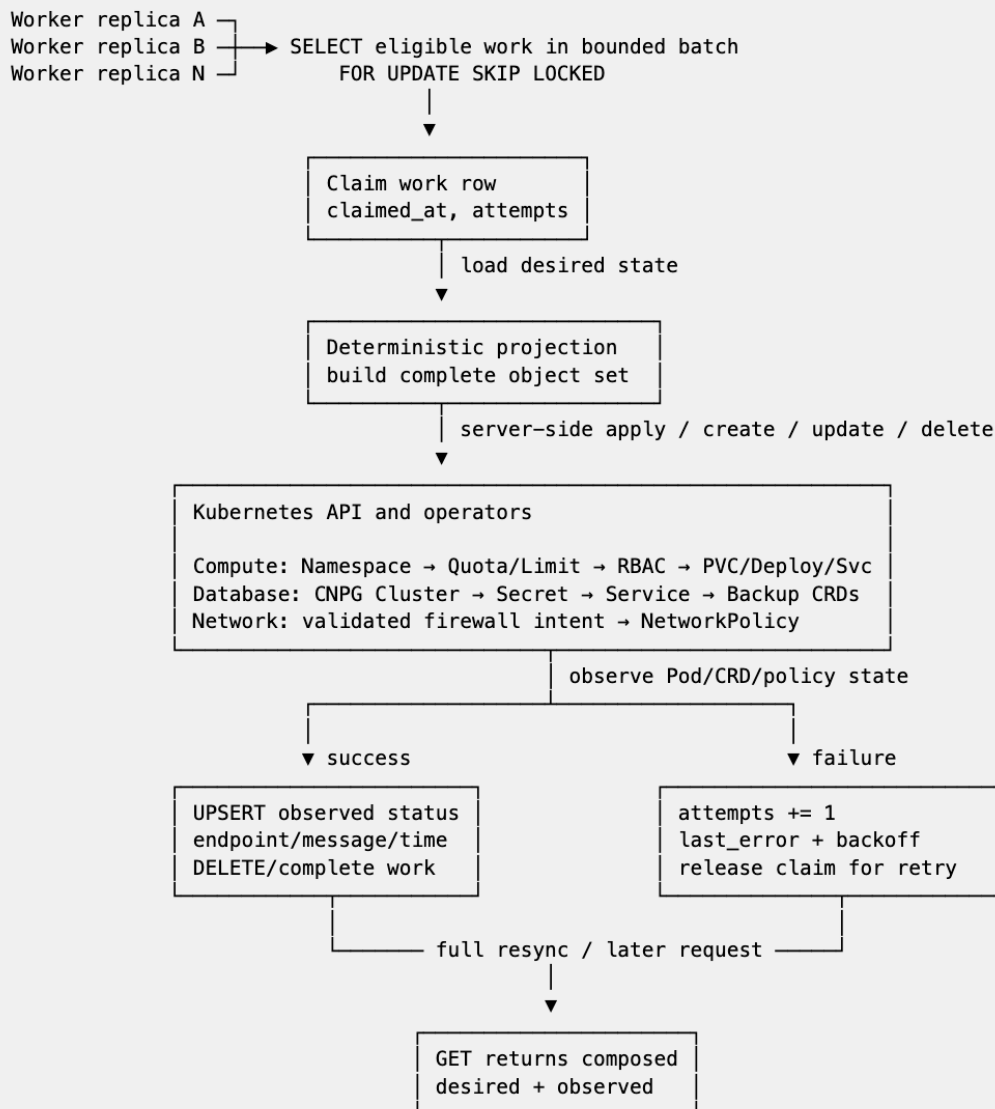


Figure 4.2: Durable desired-state acceptance and asynchronous reconciliation.

Desired and Observed State

Compute and database resources distinctly differentiate the requested lifetime from runtime monitoring. A compute-engine row logs `desired_status` as either `running` or `stopped`; a one-to-one status row documents `running`, `stopped`, `pending`, or `failed`, in addition to address, message, node, and reconciliation time. The API consolidates these fields into a consistent public status and presently associates an internal failed observation with `pending`, averting a temporary infrastructure failure from evolving into an unsupported client state. Database resources utilize identical desired-state parameters and uphold the observed status, host, port, connection metrics, backup integrity, messages, and the most recent reconciliation timestamp. Storage networks incorporate `enforcement_status` with values of `enforced`, `partial`, or `not-enforced` to ensure the API does not assert the efficacy of a firewall rule when Kubernetes is unable to depict it.

This approach differentiates acceptance from fulfillment. A successful creation or modification indicates that the authenticated intent has been verified and executed; it does not suggest that a pod is now ready, a PVC is allocated, or a CNPG primary is accessible. Clients either poll or invalidate their query status and display `pending` until the observation stabilizes.

Transactional Work Creation and Claims

The possessing service records the resource alteration and its queue entry within a single PostgreSQL transaction. A failure cannot therefore result in an acknowledged resource lacking a persistent trigger. Worker replicas acquire limited batches using `FOR UPDATE SKIP LOCKED`, allowing simultaneous workers to bypass rows currently locked by another worker rather than serializing the whole queue. Claim details, attempt frequencies, recent failures, and enqueue duration facilitate retry and oversight.

The queue operates as an at-least-once mechanism instead of an exactly-once delivery framework. A procedure could encounter failure subsequent to the implementation of a Kubernetes object however before the removal of its operational row. Consequently, it is imperative that replaying that object is secure. Laborers fabricate resources in a predictable manner and employ declarative applications, stable identifiers, current-object examination, and database uniqueness restrictions. Regular comprehensive resynchronization rectifies overlooked triggers and discrepancies, even in instances when a queue item has been lost, duplicated, or depleted.

Reconciliation by Domain

The computational worker implements account-specific requirements prior to workload entities. `ResourceQuota` and `LimitRange` delineate the capacity limits of the namespace; RBAC and pull credentials facilitate functionality; the PVC is protected from concurrent initial provisioning; and Deployment and Service entities actualize the engine. The observation captures the pod's status and logs the node, IP address, PVC condition, and messages. Soft deletion modifies the persistent state prior to the worker dismantling Kubernetes resources. A namespace reaper identifies neglected account

namespaces, employing a dry-run mode as the default precaution. Backup scheduling constitutes an independent cycle that generates restricted data tasks and tracks backup lifecycles.

The database operator requests the compute service to verify the account namespace over a distinct authorized internal endpoint. It subsequently generates a `CNPG Cluster`, an optional `Barman ObjectStore`, and `ScheduledBackup` resources, while using CNPG-generated connection `Secrets` just when necessary. The database of the platform includes the `Secret` name, namespace, and owner role but excludes the client password. Observation documents the CNPG endpoint and its functioning status. SQL execution and import are synchronous service activities that adhere to lifecycle reconciliation.

The storage network operator examines the whole network together with its sequential firewall regulations, ensures that the projection does not expand access, and reconstructs the full `NetworkPolicy` collection. Regulations on permitted TCP/UDP formats can be implemented. The Kubernetes object excludes ICMP semantics that cannot be properly expressed and is reflected in the incomplete status. Storage further functions as an autonomous consumption aggregator. It asserts the least-recently assessed buckets using row locking, examines the designated `Garage` prefix, and retains time-series use snapshots for quota estimation.

IAM employs a comparable yet unique reconciliation framework for Authentik. Local IAM modifications are executed prior to the remote administrative invocation to ensure the platform database retains its authoritative status. A failing distant synchronization results in diminished audit evidence, while a background drift reconciler evaluates local users and Authentik groups under an advisory lock. This represents eventual cross-system coherence instead of a distributed transaction.

Failure Mitigation Without a Message Broker

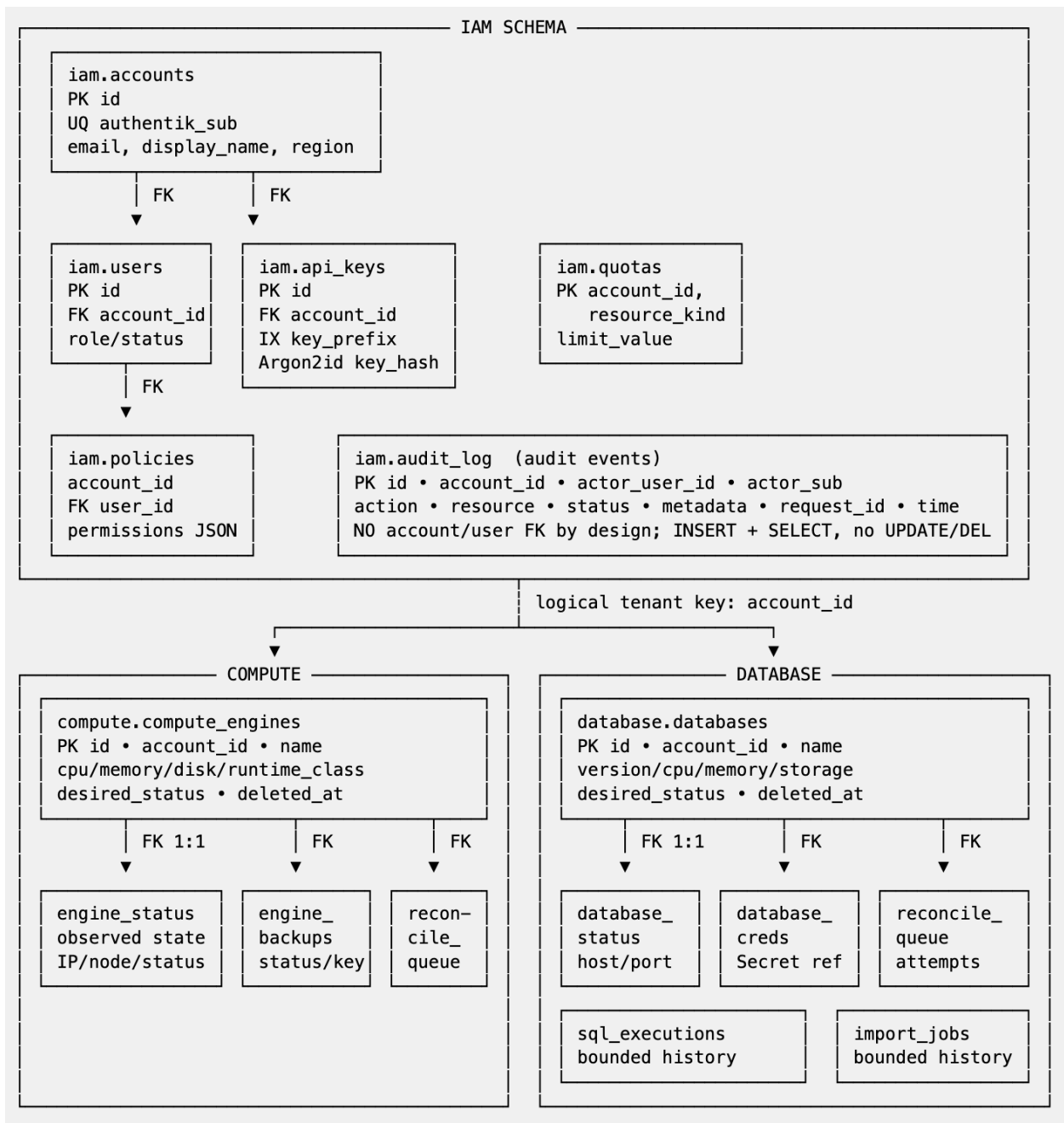
FCI intentionally steers clear of RabbitMQ, Kafka, or any other substantial broker. PostgreSQL is essential for maintaining a resilient control-plane state and offers transactions, row-level locks, advisory locks, partial indexing, and visible queue tables. Utilizing it again diminishes memory use, operational interdependencies, TLS and credential exposure, as well as failure scenarios on the Raspberry Pi cluster.

This choice is suitable for the platform's constrained workload, although it possesses a scaling limitation. Queue traffic allocates database resources for API transactions, and PostgreSQL is not designed to supplant a high-throughput event log. The architecture alleviates issue by employing short batches, backoff strategies, per-domain schemas, connection restrictions, indexed unclaimed rows, frequent resynchronization, and limited worker concurrency. A committed broker would only be warranted if the assessed queue volume, isolation prerequisites, or cross-cluster event dissemination beyond the safe capabilities of PostgreSQL.

4.3 Data Architecture & Schema Design

Platform metadata is stored in one PostgreSQL 16 cluster operated by CNPG. Service ownership is separated by PostgreSQL schema and role: `iam`, `compute`, `database`, and `storage`. Figure 4.3 presents the core relational entities. Solid lines marked `FK` are database-enforced foreign keys. Dashed

account_id lines are logical tenant references enforced by authenticated service queries rather than cross-schema foreign keys.



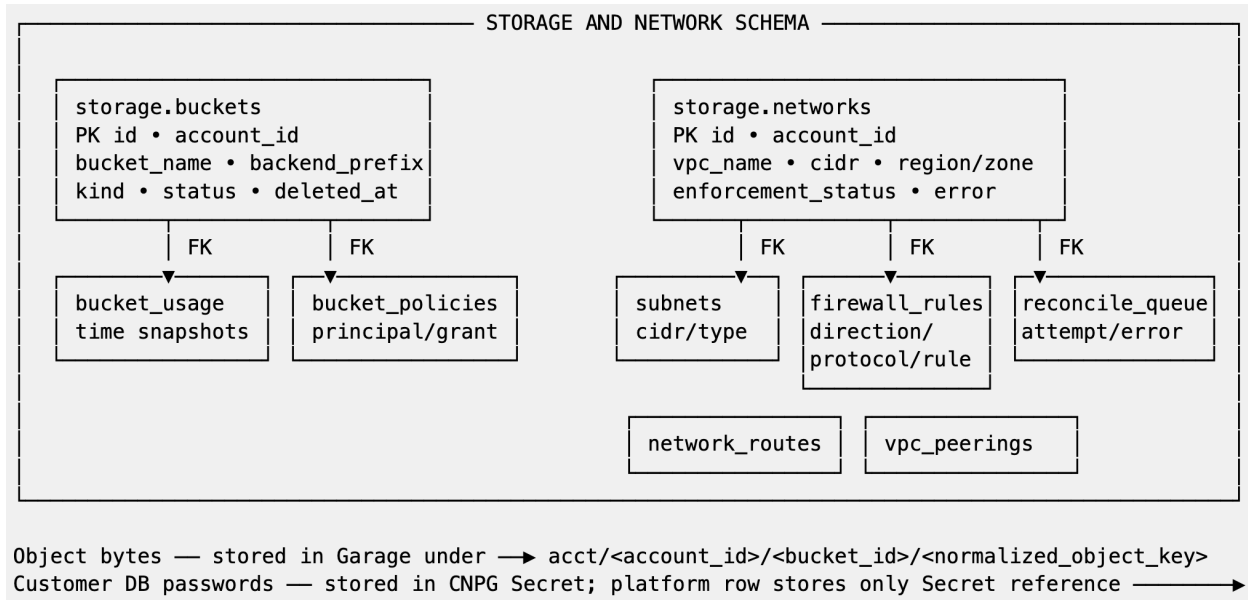


Figure 4.3: Cross-schema entity relationships and external data references.

Tenant Keys and Referential Integrity

In the IAM schema, `users`, `api_keys`, `quotas`, and `policies` utilize database-enforced foreign keys linked to `iam.accounts`. In the computational, database, and storage frameworks, `account_id` is deliberately a logical foreign key instead of a tangible reference to `iam.accounts`. This maintains service ownership and averts a migration, outage, or deletion in IAM from establishing cross-schema write interdependence. Tenant segregation is thus implemented through two methods: the gateway provides a signed account identity that clients cannot pick, and each domain query incorporates that `account_id` when accessing or altering a resource. Cross-tenant reads often yield a "not found" response instead of "disallowed," so they prevent the API from verifying the existence of another tenant's resource.

Cross-schema tenant isolation is upheld as a stringent security principle validated by service and integration testing. In every domain schema, local child relationships are maintained by the database: status, queue, backup, use, policy, subnet, and firewall entries are derived from the resource that has them.

Audit retention constitutes a purposeful deviation within IAM. The `iam.audit_log` lacks a foreign key to either account or user, as the removal of either principal should not obliterate the documentation pertaining to that deletion or any prior event. The row preserves both a user identity and the external actor subject. The production IAM role is granted `INSERT` and `SELECT` permissions, while `UPDATE` and `DELETE` permissions are denied, establishing an append-only functionality as a database privilege constraint rather than merely an application practice. Subsequent migrations broaden the authorized status configuration to document impaired external synchronization alongside both successes and failures.

API-Key Storage and Lookup

An API key consists of a public prefix and a secret with high entropy. IAM retains the key prefix, the final four characters for visualization, and an Argon2id hash; the original value is provided just once during creation and cannot be retrieved. Verification uses the prefix index to identify active candidates, then does an Argon2id comparison. A placeholder hash path is utilized when no candidate is available, ensuring that response time does not indicate if a prefix is registered. Revocation is indicated by `revoked_at`, and partial indexes omit revoked keys from the standard search route.

API-Key Storage and Lookup

Computational and database resources utilize three interconnected kinds of persistence: the requisite resource row, a one-to-one observed status row, and a queue of outstanding reconciliation efforts. Maintaining a distinction between status and specification prevents Kubernetes observations from superseding the intended configuration. Queue rows may be retried or removed without affecting the resource. Calculate the initiation and completion timestamps, status, size, target object, expiration, and errors of backup rows, using a partial unique index that permits only one pending or active backup per engine.

Database backups are orchestrated declaratively via Kubernetes and CNPG CRDs (`ScheduledBackup` and `ObjectStore`), with the backup status and health included in `database_status` instead of being preserved in a separate relational table.

Storage metadata is distinct from object data. PostgreSQL retains the logical bucket, server-generated backend prefix, use instances, and policy metadata. The garage houses the contents of the thing. In a like manner, `database_credentials` retains only the name and namespace of the Secret handled by CNPG. These reference models exclude high-value customer information from the communal platform structures.

Schema and Migration Ownership

Every service performs goose migrations for its respective schema via the common `platform-common/storage` package. The PostgreSQL connection establishes the desired schema search path, while an advisory migration lock inhibits two service replicas from executing the same migration simultaneously. Kubernetes-managed database roles provide just the schemas and functions necessary for each service. This configuration offers operational efficiency (utilizing a single PostgreSQL cluster on limited hardware) while preventing the consolidation of application ownership into a singular, unconstrained database role.

4.4 API & Interface Design

The public interface is resource-oriented HTTP JSON with one WebSocket exception. Figure 4.4 shows the implemented request lifecycle and the translation from an external credential to an audience-specific internal identity.

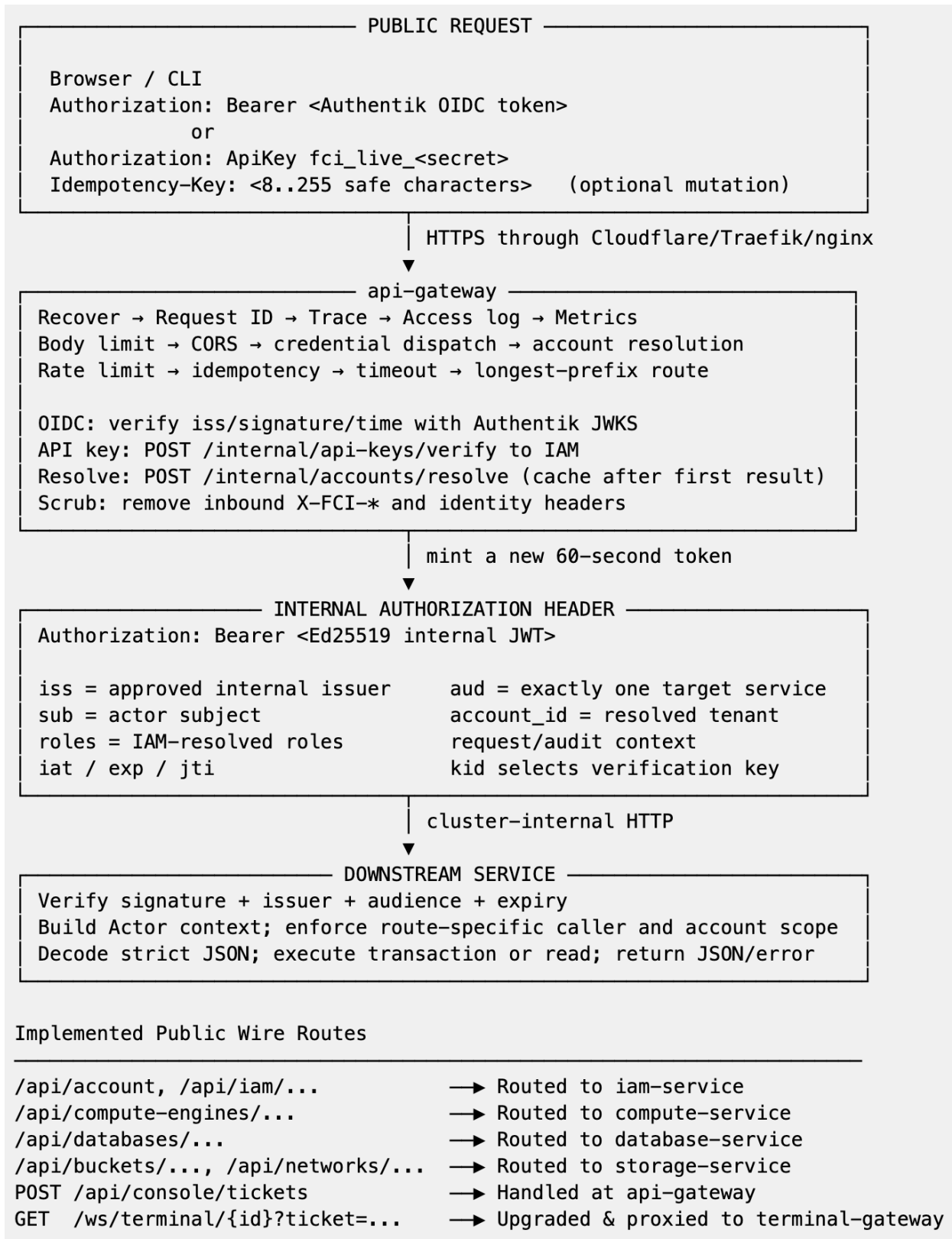


Figure 4.4: HTTP request processing, credential translation, and route taxonomy.

Resource Paths and Method Semantics

The public API uses clean, domain-oriented paths that conceal internal Kubernetes service names from clients:

- `/api/account`, `/api/account/api-keys`, and `/api/iam/users` are routed to IAM.
- `/api/compute-engines` and nested metrics and backup paths are routed to `compute-service`.
- `/api/databases` and nested metrics, connections, SQL execution, and import paths are routed to `database-service`.
- `/api/buckets` and nested object, file, metrics, and access-policy paths are routed to `storage-service`.
- `/api/networks` and nested firewall-rule paths are also routed to `storage-service`.
- `POST /api/console/tickets` is implemented at the gateway, and `/ws/terminal/{id}` is upgraded and proxied to `terminal-gateway`.

Internal routes use `/internal/...` and are not exposed as general product APIs. They include account resolution, API-key verification, quota retrieval, internal user lookup, compute namespace assurance, compute exec-target resolution, audit ingestion, and reserved backup-bucket creation. Each internal route has an explicit issuer set; possession of any valid platform token is not sufficient to call every internal interface.

HTTP methods adhere to resource semantics: `GET` retrieves, `POST` generates or triggers an action, `PATCH` modifies mutable configurations or intended states, `PUT` allocates an object to a specified logical container, and `DELETE` performs a soft deletion of a control-plane resource or removes an object/policy/rule. The successful creation yields a JSON object containing the newly created resource, updates provide either the updated representation or no content as dictated by the handler, and removals result in an empty success response when applicable. Helpers convert nil slices into empty JSON arrays, ensuring the SPA obtains a consistent collection type.

Request bodies are parsed with a size limitation, unrecognized fields are discarded, and a body must encompass precisely one JSON item. These regulations render a client/schema discrepancy apparent instead than covertly eliminating data. Domain handlers impose stricter constraints on object uploads, SQL queries, imports, pagination, naming conventions, CIDRs, and resource dimensions.

Error Contract & Response Envelopes

All Go services implement a unified JSON error envelope providing structured machine classification, descriptive messaging, request tracing, and contextual details:

```
```txt
```

```
HTTP/1.1 409 Conflict
```

```
Content-Type: application/json; charset=utf-8
```

```
{
 "error": {
 "code": "quota_exceeded",
 "message": "compute_engines quota exceeded (3 of 3 used)",
 "request_id": "correlation identifier",
 "details": {"limit": 3, "current": 3}
 }
}
...
```

Standard error codes include `invalid_input`, `unauthenticated`, `forbidden`, `resource_not_found`, `conflict`, `quota_exceeded`, `rate_limited`, `upstream_timeout`, `upstream_unavailable`, and `internal_error`. Unhandled internal failures are logged with full stack traces and correlation IDs while returning sanitized, opaque error messages to clients to avoid information leakage.

## Idempotency and Retry Behavior

The gateway permits an optional `Idempotency-Key` for `POST`, `PATCH`, and `DELETE` requests. Keys undergo validation and scoping by account prior to their storage in Valkey. The initial request asserts ownership of the key; a concurrent duplicate encounters a conflict; and a subsequent duplicate is capable of replaying the recorded status, headers, and body. Responses that encounter a server error or over the capture threshold are not preserved for the whole replay duration.

This method serves as an enhancement rather than a definitive assurance of singular occurrence. Valkey exhibits non-persistence, its cache mechanisms default to an open failure mode, and an entry is subject to eviction. Accuracy therefore persists in service transactions, uniqueness stipulations, and idempotent reconciliation. Gateway retries are constrained to secure failure scenarios prior to receiving an upstream response and are regulated by individual upstream circuit breakers and time limits. WebSocket traffic is never automatically retried.

## WebSocket Terminal Interface

The terminal route employs `GET /ws/terminal/{id}?ticket=...` as browser WebSocket APIs are unable to incorporate a custom `Authorization` header. Following the update, text frames from the client are considered outdated raw terminal input. Binary frames can utilize `0x00` for input or `0x01` succeeded by a JSON resize directive like `{"type": "resize", "cols": 220, "rows": 50}`. Dimensions are constrained within the range of 1 to 1,000 and can be adjusted by configuration; resize surges are mitigated. The server's output is sent in unprocessed text format. A warning frame prefixed with `0x02` indicates an impending idle timeout, maximum duration, or server drainage. The ping/pong protocol ensures liveness and rejuvenates the Valkey session-slot TTL.



## 4.5 Security, Authentication & Access Control

FCI implements a zero-trust framework across multiple autonomous layers: the public credential is authenticated at the perimeter; public identity headers are eliminated; a transient credential is generated for a specific internal audience; downstream pathways authenticate sanctioned issuers; account identity is reinstated for each query; Kubernetes RBAC limits service accounts; NetworkPolicies restrict network accessibility; and secrets are instantiated solely within the pods that necessitate them.

Figure 4.5a shows the two external HTTP authentication paths and their convergence on internal service identity.

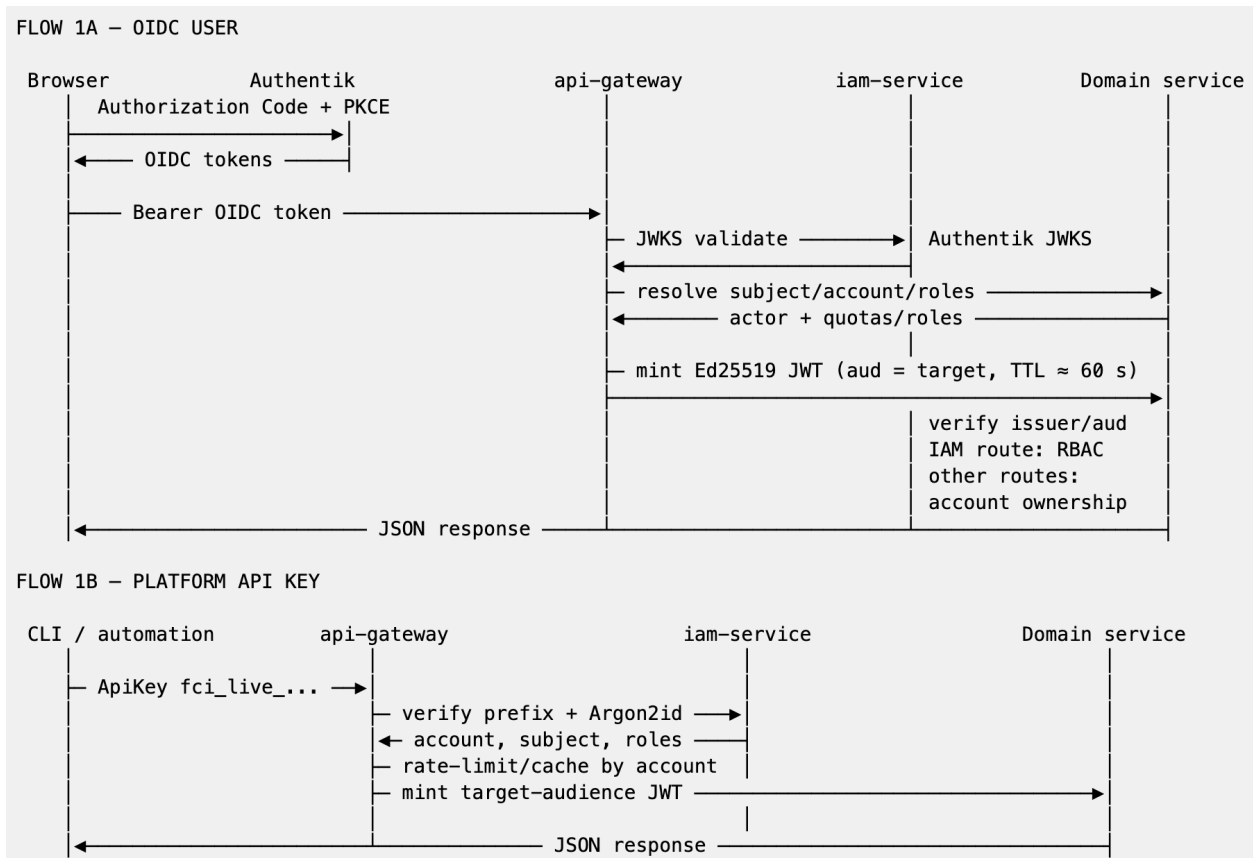


Figure 4.5a: OIDC and API-key authentication translated into internal service identity.

Figure 4.5b shows the separate ticket exchange used by browser terminals.

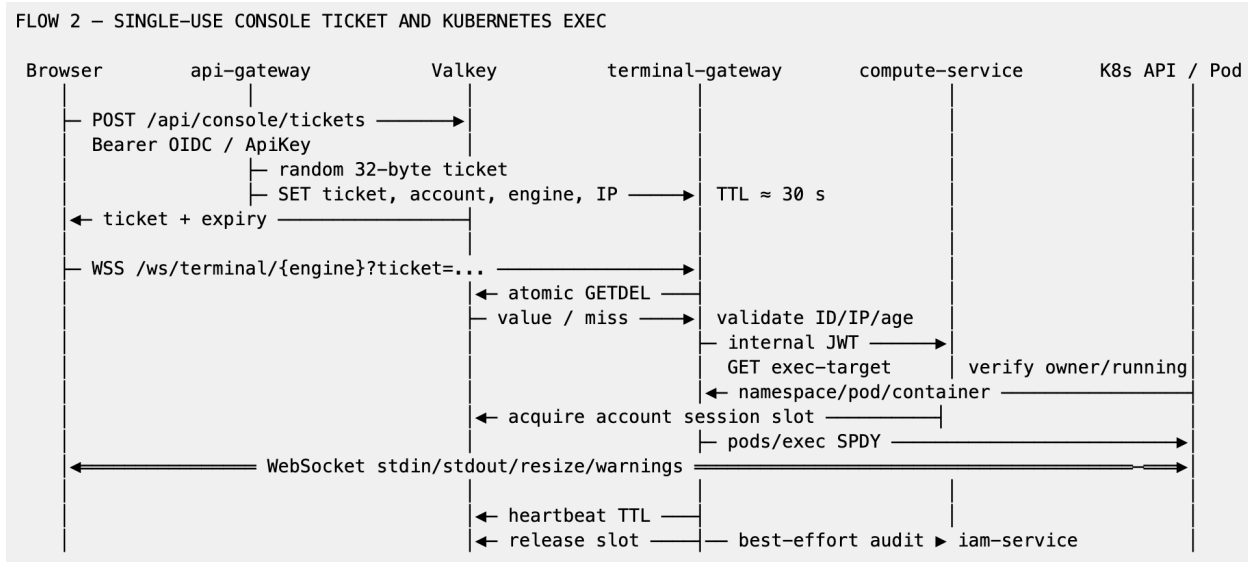


Figure 4.5b: Single-use terminal authorization and WebSocket-to-pods/exec bridge.

## Authentication and Authorization Separation

Authentik serves as the external identity supplier. It verifies the identity of the browser user and generates OIDC tokens; it does not possess FCI resource allocations or product audits. IAM associates the stable OIDC topic with an account and local user, establishes the initial owner and default quota, retains managed and custom rules, and synchronizes the chosen user/group status back to Authentik following a local commit. API keys signify the credentials for account automation and now possess the whole administrative role configuration of the account.

Roles are determined by IAM and included in the signed token, ensuring that an IAM permission decision does not necessitate a database query for each request. IAM employs a deny-overrides policy assessment and a default deny approach for its safeguarded product operations. Compute, database, and storage predominantly uphold authenticated tenant ownership and quotas instead of reconstituting the IAM policy framework. Internal operational pathways incorporate caller identification: the database service may access the namespace endpoint, the terminal gateway only may invoke the exec target, the compute service might secure a designated backup bucket, and some services may acquire quotas or user records.

## Internal JWT Trust Boundaries

Internal tokens are authenticated with Ed25519 signatures and typically have a validity period of 60 seconds. Every target authenticates the signature, `exp` and temporal assertions, anticipated audience, and a permissible issuer-to-public-key correspondence. The audience consists solely of ``iam-service``, ``compute-service``, ``database-service``, ``storage-service``, or ``terminal-gateway``. This inhibits a token generated for a low-risk pathway on one platform from being reused on another. Distinct key pairs are utilized for gateway and service identities engaged in direct communications. The gateway eliminates all

incoming FCI and identity headers prior to appending its own authorization context, so preventing a client from illicitly transmitting a fraudulent account identifier across the proxy.

The WebSocket route does not include a bearer token within its query parameters. Query parameters are often preserved in browser history and access logs, however browser WebSocket APIs are unable to establish a custom permission header. The 30-second ticket is therefore a limited functionality: it is arbitrary, associated with a single engine and account, optionally linked to the source address, removed immediately upon initial utilization, and thereafter verified by a compute ownership assessment. If Valkey is not accessible, redemption is unsuccessful and terminated. Session-cap acquisition could fail to close because it serves as a resource-management enhancement rather than a security threshold.

## OpenBao and Secret Materialization

OpenBao is deployed prior to Argo CD via Ansible, as External Secrets is unable to authenticate with a secret store that has not been started and unsealed. The bootstrap procedure generates TLS assets, initializes and unseals the replicas, activates the necessary secret and Kubernetes-auth pathways, provisions application credentials and Ed25519 keys, and establishes least-privilege rules. The bootstrap token is an out-of-band operator secret and is not recorded in Kubernetes.

Upon the installation of External Secrets and its configuration by Argo CD, a `ClusterSecretStore` establishes authentication with OpenBao via Kubernetes authentication. `ExternalSecret` resources, typically updated on an hourly basis, replicate solely designated properties into Kubernetes Secrets that are scoped to the namespace. `Workload Deployments` incorporate such `Secrets` as files or environmental variables. Private signing keys, public verification keys, PostgreSQL credentials, Garage credentials, Valkey credentials, and CA certificates are allocated based on service requirements. Containers operate as non-root entities, relinquish Linux powers, utilize read-only root filesystems where feasible, and mount secret files in a read-only manner. Gateway and IAM do not enable automated service-account token mounting as they do not interact with the Kubernetes API; compute, database, storage, and terminal are granted just the Kubernetes RBAC necessary for their control-plane operations.

## Object and Database Data Isolation

The storage service never acknowledges a full Garage object key from the client. It standardizes the specified object name and generates the physical key in the format `acct/<account_id>/<bucket_id>/<object_key>`. Validation disallows erroneous UTF-8, control characters, traversal, absolute paths, and any other formats that could circumvent the prefix. Bucket IDs are retrieved from the authenticated account prior to the formation of the prefix. The garage is not accessible to the public, and renters are not provided with the shared backend credentials or presigned direct URLs.

Bucket access-policy regulations authenticate principals using IAM at the application tier. The principal object isolation boundary is upheld by ``storage-service`` via rigorous request authentication and storage prefixes obtained from tenants prior to assigning tasks to Garage.

Customer database passwords adhere to a distinct isolation barrier. CNPG produces and retains them in a Secret located inside the account namespace. The database service retrieves the Secret in real-time while establishing a limited per-database connection pool and does not retain the password within the PostgreSQL platform. Namespace RBAC restricts the service accounts that can access or manipulate client resources.

## Kubernetes Network and Workload Boundaries

Application NetworkPolicies restrict inbound traffic to designated namespaces and ports. The frontend receives incoming traffic from Traefik; the gateway processes proxied application requests and communicates solely with its specified backends, Authentik/JWKS, and Valkey; backend services permit access from the gateway or specifically authorized internal callers. Client workloads are allocated under `fci-cust-<account-uuid>` namespaces, incorporating ResourceQuota, LimitRange, RBAC, and duplicated image-pull credentials.

User-specified VPC and firewall settings are directly converted into Kubernetes NetworkPolicies. The `storage-service` verifies CIDR allocations in relation to cluster ranges and account limits, translating permissible allow-rules into native NetworkPolicy entities while designating unsupported protocols as unenforced. Network regulations rigorously mandate ingress segregation between tenant and platform namespaces, while egress policies are governed by configuration parameters.

## Security Failure Posture and Auditability

FCI differentiates between dependencies that are important for accuracy and those that serve as performance enhancements. Authentication failures, unreliable issuers, incorrect audiences, PostgreSQL access denials, terminal-ticket repository errors, inter-account queries, and inaccessible dedicated-runtime requirements result in a closed failure. Valkey-supported caches, standard rate-limiting storage, and idempotency assertions could fail in an open state, ensuring that the loss of temporary acceleration does not render the control plane inaccessible; service database limitations persist as the border of correctness.

Request identities, organized logs, analytics, and OpenTelemetry context facilitate incident correlation. IAM mutation pathways are documented with an anticipated audit, and initialization or testing are unsuccessful if a new mutation remains unclassified. Audit logs preserve details such as actor, account, action, resource, status, metadata, request identifier, and timestamp, but database permissions inhibit the IAM application role from modifying or erasing them. Collectively, these mechanisms provide security determinations visible across the edge, domain, data, and Kubernetes levels without supposing that mere network location confers confidence.

## 5. DevOps, Infrastructure & Delivery Pipeline

FCI's operational framework distinguishes between the initial setup of machines and the ongoing reconciliation of clusters. Ansible executes the privileged, singular transition from seven Raspberry Pi operating system installations to a fully operational K3s and secret management framework. Terraform

specifies the external perimeter and discretionary cloud assets. GitHub Actions authenticates and disseminates artifacts, whereas Argo CD regards Git as the definitive intended state for consistent operational performance. This separation ensures that the day-two platform does not rely on recurring imperative commands and allows for infrastructure modifications to be assessed across separately versioned repositories.

## 5.1 Bare-Metal Bootstrap & Ansible Automation

The `ansible-automation` repository owns the physical-cluster bootstrap. Its `playbook.yml` targets the master and worker inventory groups in dependency order, retrieves the K3s join token from the first server, and delegates detailed operations to idempotent roles. Figure 5.1 shows the boundary between this bootstrap and the GitOps-controlled steady state.

Operator / CI control node

|  
ansible-playbook playbook.yml  
v

1. Validate inventory and prepare seven Raspberry Pi ARM64 nodes
  - └ configure boot/cgroup prerequisites and required OS packages
  - └ optionally join Tailscale for administrative connectivity
2. Install the K3s server plane
  - └ first master: --cluster-init
  - └ remaining masters: join the server plane
  - └ retrieve the shared node token and verify readiness
3. Install K3s agents on worker nodes
  - └ join through the first master and retrieved token
  - └ label/taint high-, mid-, low-memory and control-plane nodes
4. Establish the pre-GitOps trust root on the first master
  - └ install OpenBao with bootstrap TLS
  - └ initialize and unseal; enable KV, audit, policy and K8s auth
  - └ seed application credentials and Ed25519 key material
5. Install and configure Argo CD
  - └ wait for controllers and the Application CRD
  - └ apply root-app pointing to k3s-manifests

| ownership hand-off  
v

Argo CD app-of-apps

- └ reconcile namespaces, operators, storage, identity and ingress
- └ materialize named OpenBao properties through External Secrets
- └ patch CA-dependent OpenBao values after cert-manager becomes ready
- └ deploy product charts; continuously prune and self-heal

|  
v

Steady-state seven-node K3s platform

Figure 5.1: Imperative bare-metal bootstrap followed by declarative GitOps ownership.

Inventory verification occurs prior to node reconfiguration. The boot setup of Raspberry Pi facilitates the necessary kernel and cgroup functionality for Kubernetes, while the K3s roles install essential packages, set up swap behavior, deploy the initial server with cluster initiation, include additional servers and agents, and monitor API health. Node labels and taints prevent control-plane and low-memory nodes from being assigned inappropriate workloads. This pertains to the creation of a targeted platform and confidential hygiene, rather than asserting comprehensive CIS operating system hardening; the

repository lacks a singular host-hardening role. An optional Kata Containers role is available for KVM-capable high-memory systems, however it is deactivated in the Raspberry Pi play because to the absence of necessary hardware virtualization.

OpenBao is deployed straight through Ansible as External Secrets cannot authenticate with a non-existent secret store. The function establishes bootstrap TLS, identifies a functional OpenBao endpoint, configures a five-share/three-threshold seal if necessary, preserves recovery materials externally from Git, and unseals the replicas. Following chores facilitate KV v2, audit logging, least-privilege protocols, and Kubernetes authentication, subsequently initializing Authentik, PostgreSQL, Garage, Valkey, GitHub registry, and Ed25519 signature materials. The bootstrap token is neither stored nor replicated within a Kubernetes Secret of the application.

Argo CD is deployed just once this trust root is functional. Ansible implements the upstream manifests, restricts Argo CD components to nodes with high memory, installs health modifications for External Secrets, awaits readiness, and generates the root Application. A concluding CA-seeding function is pending for cert-manager's self-signed CA, which only modifies the CA-dependent OpenBao attributes for Valkey, Garage, and platform PostgreSQL. The configuration of the local Kubeconfig and K9s finalizes the operator workflow while preserving the cluster's stable source of truth.

## 5.2 Containerization & Multi-Arch Build Strategy

Each Go service employs a multi-stage Docker build. A Go 1.26 Alpine builder retrieves modules via BuildKit cache mounts, acquires the ephemeral private-module credential as a secret mount, and builds using `CGO_ENABLED=0, GOOS=linux, GOARCH=arm64, -trimpath`, and minimized linker flags. The gateway, IAM, database, storage, and terminal images solely transfer the static binaries into `distroless/static-debian12:nonroot`. The `compute-service` utilizes `scratch` and specifically duplicates the CA bundle necessary for outbound TLS. Both variants operate under the non-root identity `USER 65532:65532`, reveal just application and metrics ports, and lack both a shell and a runtime package management.

The frontend is also multi-tiered. Node 22 generates architecture-agnostic Vite assets on the build host, circumventing the instability seen while executing Node under ARM emulation. The execution environment is specifically `linux/arm64 nginx-unprivileged`, operates under UID 101, listens on port 8080, and features a minimal `/healthz` verification. The Nginx template offers SPA fallback, security and cache headers, `/api/` proxying, and WebSocket upgrade forwarding, whereas Kubernetes dynamically injects environment-specific `/config.js` variables during runtime.

Docker Buildx provides a build system that supports several platforms, and frontend smoke tests utilize QEMU when the architecture of the runner diverges from that of the target. The existing publication process intentionally chooses just `linux/arm64`, as all production nodes operate on ARM64. It should thus be defined as an ARM64 cross-build methodology rather than a disseminated multi-architecture manifest list. Incorporating `linux/amd64` thereafter necessitates validating every Dockerfile and releasing a unified OCI index; Buildx offers the functionality, although the evaluated release artifacts currently do not assert that compatibility.

## 5.3 Continuous Delivery Workflows (GitHub Actions)

Every repository encompasses a minor caller workflow that assigns tasks to reusable processes located in the organization's `.github` repository. This configuration consolidates policies while preserving autonomous repository versioning. Go repositories execute `go-pr-checker.yml`. The validation graph verifies `go mod tidy`, formatting, generated `sqlc` [32] discrepancies, and `golangci-lint` [34]; conducts race-enabled unit and integration tests with coverage; reiterates chosen tenant-isolation tests; executes repository-specific data-loss, `envtest`, `Kind`, or end-to-end objectives; uploads coverage data for SonarQube; carries out `govulncheck` and `Semgrep` analyzes; and performs smoke builds of ARM64 containers when a `Dockerfile` exists.

Every repository encompasses a minor caller workflow that assigns tasks to reusable processes located in the organization's `.github` repository. This configuration consolidates policies while preserving autonomous repository versioning. Go repositories execute `go-pr-checker.yml`. The validation graph verifies `go mod tidy`, formatting, generated `sqlc` discrepancies, and `golangci-lint`; conducts race-enabled unit and integration tests with coverage; reiterates chosen tenant-isolation tests; executes repository-specific data-loss, `envtest`, `Kind`, or end-to-end objectives; uploads coverage data for SonarQube; carries out `govulncheck` and `Semgrep` analyzes; and performs smoke builds of ARM64 containers when a `Dockerfile` exists.

The frontend triggers `frontend-pr-checker.yml`, which executes `npm ci`, `Oxlint`, rigorous TypeScript compilation, the Vite production build, Vitest coverage assessment, SonarQube evaluation, and an ARM64 container smoke test. Ansible validation independently confirms that the Vault file is still encrypted. Concurrency groups terminate outdated executions for a branch, preserving the finite resources of self-hosted runners.

Artifact publication is explicitly dispatched through each repository's `build-and-push.yml`, which calls the organization workflow of the same name. Docker metadata assigns a long commit-SHA tag by default, accepts an operator-provided release tag, and assigns `latest` only for the default branch. BuildKit receives the GitHub App token only as a secret mount, preventing it from becoming an image layer. All container images are published directly to GitHub Container Registry (`ghcr.io`).

Image publication does not automatically rewrite deployment manifests. Promotion is a separate, reviewed change to `image.tag` or `image.digest` in the GitOps repository (`k3s-manifests`). This separation establishes an explicit release audit trail and ensures that builds are deliberately promoted rather than automatically deployed to production.

## 5.4 Declarative GitOps: Argo CD App-of-Apps & Sync Waves

Figure 5.4 presents the complete multi-repository delivery path. CI determines whether source is admissible and produces an image, but Git remains the authority that selects which artifact is deployed.



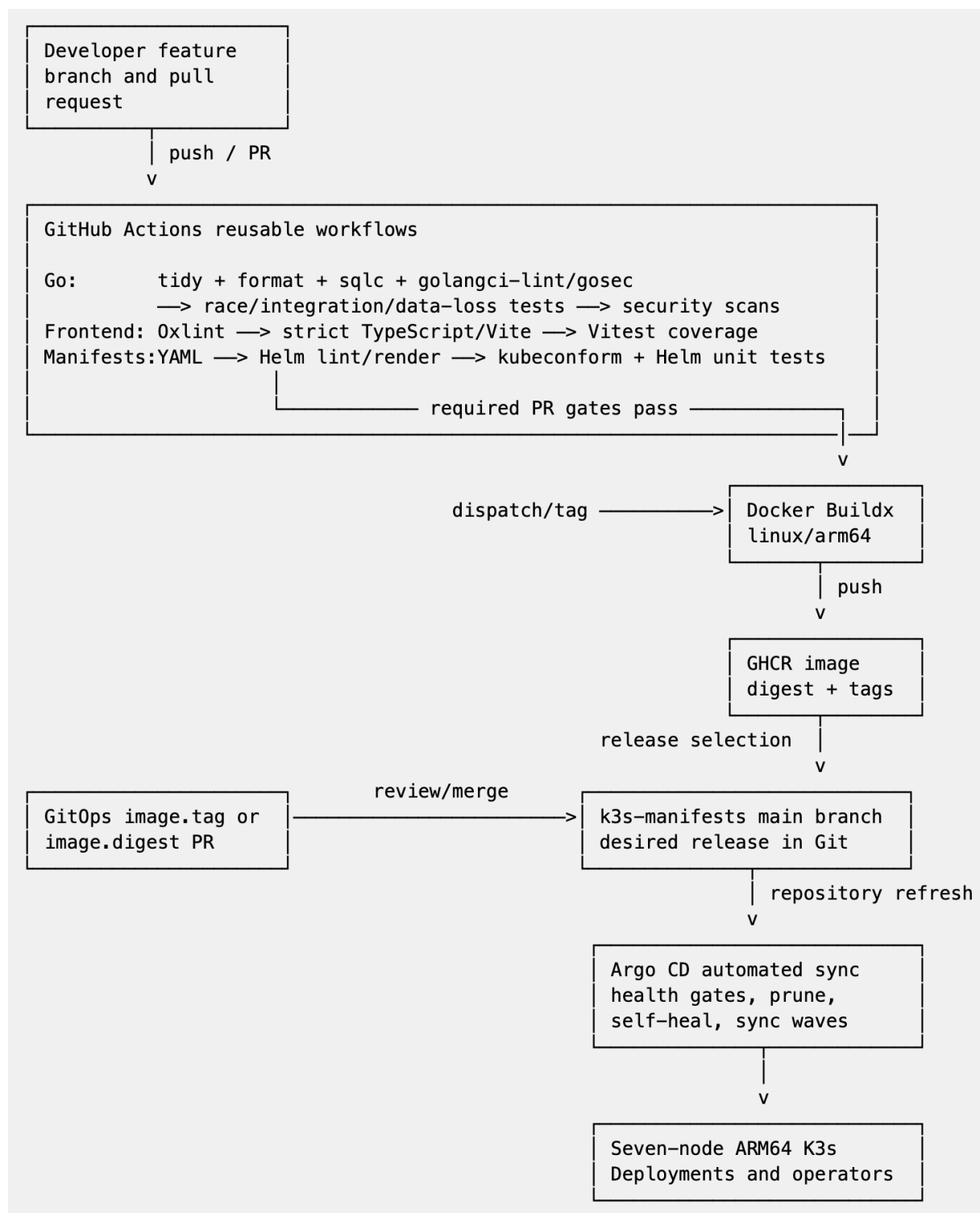


Figure 5.4: Multi-repository validation, image publication, and GitOps promotion flow.

The Ansible-generated root Application systematically identifies infrastructure and product subordinate Applications from `k3s-manifests`. Automated synchronization facilitates pruning, self-repair, empty-source protection, namespace generation, server-side application if necessary, and dry-run

compatibility for resources without existing CRDs. Child Applications integrate external charts with values held by the repository or implement local FCI charts. Tailored health evaluations guarantee that a subsequent wave does not advance only due to the acceptance of a `ExternalSecret` or operator resource by the API server.

The following shortened Application preserves the implemented release pin and ordering mechanism:

```
``yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
 name: api-gateway
 namespace: argocd
 annotations:
 argocd.argoproj.io/sync-wave: "11"
spec:
 source:
 repoURL: https://github.com/freecloudinitiative/k3s-manifests.git
 targetRevision: HEAD
 path: applications/api-gateway
 helm:
 parameters:
 - name: image.tag
 value: "v0.5.5"
 syncPolicy:
 automated: { prune: true, selfHeal: true }
 syncOptions: [CreateNamespace=true, ServerSideApply=true]
``
```

The repository implements a dependency sequence from wave 0 through wave 11:

**Wave 1:** Foundational controllers. cert-manager [16], External Secrets Operator, and Kyverno [18] introduce certificate, secret-materialization, and admission-policy capabilities.

**Wave 1:** Foundational controllers. cert-manager, External Secrets Operator, and Kyverno introduce certificate, secret-materialization, and admission-policy capabilities.

**Wave 2:** Storage, database, policy, and log foundations. CloudNativePG, Longhorn, Loki, and Kyverno policies become available. `ExternalSecret` instances use internal wave 2 after their operator and `ClusterSecretStore` exist.`

**Wave 3:** Controller configuration. Certificate issuers and External Secrets store configuration are applied after the corresponding CRDs and controllers become healthy.

**Wave 4:** Persistent platform backends. Garage, MetalLB, platform PostgreSQL, and Valkey are reconciled. Garage and PostgreSQL manifests use sub-waves to order bootstrap jobs, certificates, roles, and databases.

**Waves 5–8:** Identity and telemetry. Authentik, MetalLB configuration, and the OpenTelemetry Collector occupy wave 5; Alloy follows at wave 6 and Tempo at wave 8.

**Wave 9:** Ingress and operational surfaces. Traefik, Cloudflare Tunnel, and the kube-prometheus-stack start after identity, certificate, storage, and telemetry dependencies.

**Wave 10:** Argo CD runtime configuration. Repository-driven Argo CD configuration is applied after the services needed by its external access and identity integration.

**Wave 11:** FCI products. The frontend, gateway, IAM, compute, database, storage, and terminal charts are released only after platform dependencies pass their health gates.

The outcome is a definitive installation and reversion system. Argo CD consistently contrasts live resources with Git, reinstates altered fields, and eliminates deleted resources. Restoring an image or manifest commit leads the identical controller route to align with the previous declaration; an independent rollback process is unnecessary.

## 5.5 Infrastructure-as-Code & Edge Automation

The `terraform-cloudflare-infra` configuration establishes the public edge without revealing a Kubernetes LoadBalancer or NodePort to the web. The Cloudflare provider establishes the root and service DNS entries, a designated Zero Trust tunnel, and sequential ingress regulations. The primary hostname and public service hostnames are proxied CNAMEs directed to the tunnel; entries designated as `internal_only` are instead unproxied. A record is direct to the LAN entrance address and are omitted from the tunnel setup. By default, every public rule directs to Traefik's in-cluster `websecure` Service, while a concluding `http_status:404` rule ensures that mismatched hostnames do not access an origin.

The produced 32-character tunnel secret and tunnel token serve as deployment credentials. Terraform establishes the tunnel and settings, while the `cloudflared` pod is independently installed by Argo CD and acquires the credentials via the platform's private pathway. The in-cluster hop bypasses origin-certificate validation, as Traefik offers a certificate for the public hostname instead of the Kubernetes Service DNS name; public TLS continues to terminate and is authenticated at Cloudflare's edge. Provider credentials are introduced via environment variables or CI secrets and are not included in committed `.tf` files. The module is set to local state for manual operations, while the reusable CI workflow has a GCS backend setup and a unique state prefix for automated planning or application.

The two multi-cloud repositories serve as optional experiments for automation and portability, rather than being essential components of the seven-node hosted architecture. The

`terraform-multicloud-infra` encompasses provider-specific foundations for GCP, AWS, Azure, Linode, and Civo. The most comprehensive modules provide ARM64 or relevant provider instances, disks, VPC or network assets, firewalls, public/private IP addresses, and basic versus high-availability node configurations appropriate for a K3s inventory. Reusable Terraform workflows execute formatting and validation for each provider directory, whereas `apply` and `destroy` are explicit operations that must be manually initiated with a distinct remote-state prefix. Deployed GCP, Azure, and Civo infrastructures to establish runner virtual machines; enforced SSH/network regulations; and configured startup templates that enroll one or several organizational runners per VM. GCP uses an Ubuntu ARM64 image and T2A machine types to align with the production build architecture. The selection of providers and the simple/HA modes serve as process inputs. The AWS and Linode directories presently offer just incomplete provider frameworks and should not be seen as comprehensive runner implementations. These repositories illustrate that CI capabilities and different cluster experiments may be replicated declaratively without altering the on-premises product design.

## 5.6 Quality Gates, Linters & Security Tooling

The multi-repository framework mandates quality agreements through uniform Make goals across Go modules. `golangci-lint` runs `errcheck`, `govet`, `staticcheck`, `revive`, `gosec`, and SQL row-error checks, while limiting concurrency due to Raspberry Pi memory limitations. Continuous integration guarantees that the produced `sqlc` bindings correspond with the queries by rejecting unclean trees, whilst integrated Goose migrations are verified against actual PostgreSQL databases during integration testing.

Testing encompasses various strata beyond mock handlers: unit suites address domain validation, policies, retry mechanisms, and protocols; Testcontainers offers transient PostgreSQL and Valkey dependencies; `envtest` assesses Kubernetes RBAC and controller projections; and Kind suites evaluate pods, storage, restoration tasks, and terminal interactions. Race identification, tenant segregation, and preventative measures against catastrophic data loss are consistently emphasized.

Frontend pipelines impose rigorous tests for TypeScript, Oxlint, Vitest, React Testing Library, MSW, code coverage, and accessibility. Kubernetes manifests and charts undergo validation using `yamllint`, `yq`, `helm lint`, `helm template`, rigorous `kubeconform` [35], Helm unit tests, and PostgreSQL connection budgeting. Terraform mandates syntax and provider verification without backends, whereas Ansible safeguards Vault encryption.

Security automation enhances functional validation: `govulncheck` detects accessible vulnerabilities, Semgrep examines static patterns, SonarQube monitors metrics, and CodeRabbit/Greptile conduct context-sensitive PR evaluations. Constructs utilize transient BuildKit secrets and ARM64 smoke examinations, while Helm charts disallow unpinned image tags. Ephemeral PR gates integrate into the primary branch, securing deployments to GitOps release commits for dependable rollbacks.

## 6. Software Implementation & Microservice Architecture

FCI operates its cloud control plane using a collection of specifically focused Go services and a React single-page application [1, 2]. Throughout the backend repositories, HTTP handlers authenticate transport input; service packages uphold product invariants; created `sqlc` queries maintain the intended state; workers synchronize external systems; and projection packages formulate Kubernetes resources [5]. Persistent tasks are retained in PostgreSQL; Valkey is confined to transient coordination [10]. This architecture allows for autonomous deployment and scalability without the need for a cumbersome message broker that would be excessive for the Raspberry Pi platform.

### 6.1 Shared Runtime (`platform-common`)

`platform-common` is a versioned Go module that consolidates inter-service agreements, middleware, and runtime elements across Go microservices. Consumers provide specific versions in `go.mod` to ensure repeatable container builds, whilst the workspace `go.work` enables synchronized local development.

The fundamental packages offer essential platform infrastructure: `auth` manages `Actor/AuditInfo` context, Authentik [15] OIDC validation, Ed25519 key management, and multi-issuer JWT verification; `httpx` provides standard middleware, error envelopes, and lifecycle probes; `storage` oversees TLS `pgxpool` connections, schema isolation, and advisory-locked Goose [33] migrations; `cache` incorporates bounded Valkey operations for rate limiting and sessions; and `obs` harmonizes `slog` logging and OpenTelemetry tracing with shared testing fixtures.

### 6.2 Edge Ingress & API Gateway (`api-gateway`)

The gateway uses Go's `net/http`, `httputil.ReverseProxy`, and a prefix registry to map public routes to upstream services. Browser access tokens are validated against Authentik's cached JWKS document, including issuer, audience, expiry, and key rotation. The stable OIDC subject is resolved through IAM into an FCI account. Machine clients present an opaque API key: its non-secret prefix narrows the IAM lookup, IAM performs the Argon2id comparison, and both mechanisms produce the same internal `Actor` representation.

The gateway then mints a short-lived Ed25519 JWT scoped to the selected service. The following abridged implementation also shows the mandatory removal of client-controlled identity headers before the trusted hop:

```
```go
```

```
claims := internalClaims{
```

```
claims := internalClaims{
```

```
    Roles: actor.Roles,
```

```
    Audit: auditClaims{AuthentikSub: actor.Audit.AuthentikSub,
```

```
        APIKeyID: actor.Audit.APIKeyID},
```

```

RegisteredClaims: jwt.RegisteredClaims{
    Issuer: issuer, Subject: actor.AccountID,
    Audience: jwt.ClaimStrings{upstream},
    IssuedAt: jwt.NewNumericDate(now),
    ExpiresAt: jwt.NewNumericDate(now.Add(ttl)),
    ID: actor.RequestID,
},
}

token := jwt.NewWithClaims(jwt.SigningMethodEdDSA, claims)
token.Header["kid"] = keyID
signedToken, err := token.SignedString(privateKey)

if err != nil {
    return err
}

proxy.StripClientHeaders(req.Header)
proxy.SetInternalAuth(req.Header, signedToken)
...

```

The Valkey limiter is more precisely an atomic fixed-window counter than a continuously refilled token bucket. A Lua script performs `INCR`, establishes `PEXPIRE`, and returns count and remaining TTL in one round trip. Limits are keyed by resolved account and route class; cache failure may fail open because authorization remains durable in IAM and PostgreSQL, while Traefik supplies an independent per-source-address boundary.

Upstream resilience combines request budgets, safe bounded retries, and an in-memory circuit breaker per replica and upstream. Consecutive transport failures or 5xx responses open the breaker, the cooldown admits one half-open probe, and success closes it. Only replay-safe or reconstructible requests are retried. Cancellation and deadlines take precedence over retry classification. Response handling removes hop-by-hop and server-disclosure headers, marks `/api/` data no-store, and propagates request and OpenTelemetry context.

6.3 Container-Native Compute Engine (`compute-service`)

`compute-service` implements engine creation, listing, inspection, start, stop, restart, resize, backup, restore, and soft deletion. Mutations validate instance types, quotas, regions, zones, images, and

runtime capability before committing desired state. Engine metadata does not claim that a pod already exists; desired status and observed Kubernetes status remain distinct.

Workers share the durable queue through a single PostgreSQL update over locked candidates. Abandoned claims become eligible after a stale interval, while `SKIP LOCKED` allows horizontal workers to proceed without duplicate ownership:

```
```sql
UPDATE compute.reconcile_queue AS q
SET claimed_at = now(), attempts = q.attempts + 1
WHERE q.id IN (
 SELECT id
 FROM compute.reconcile_queue
 WHERE enqueued_at <= now()
 AND (claimed_at IS NULL
 OR claimed_at < now() - $1::text::interval)
 ORDER BY enqueued_at
 FOR UPDATE SKIP LOCKED
 LIMIT $2
)
) RETURNING q.id, q.compute_engine_id, q.attempts;
```
```

The claim transaction commits before Kubernetes I/O. A worker loads current desired state, builds a deterministic projection, applies it, observes the workload, and persists status. Success deletes the queue row; transient failure clears the claim and reschedules `enqueued_at` with bounded backoff. Periodic resynchronization enqueues live engines missing from the queue and therefore repairs drift without a new API mutation.

The Kubernetes client derives ``fci-cust-<account-uuid>`` and ensures the `Namespace`, `ResourceQuota`, `LimitRange`, service accounts, role bindings, and image-pull `Secret`. Per engine it projects a `PVC`, `Deployment`, and `Service` with stable ownership labels. Typed `client-go`, a dynamic client, and controller-runtime server-side apply are used according to resource type and CRD needs. Field ownership is assigned to the FCI manager. A PVC safety marker prevents the reconciler from silently recreating an empty disk when a previously provisioned claim disappears.

Stopping scales the `Deployment` and removes pods while retaining the disk. Soft deletion records `deleted_at`; cleanup removes projected resources and finalizes the database record only after deletion is observed. Backups use durable metadata plus Jobs that archive to an account-reserved Garage bucket, record checksums, and acquire restore locks to prevent concurrent mutation of a PVC. Automated scheduling is reconciler-driven rather than dependent on an external message system.

6.4 Managed PostgreSQL Engine (`database-service`)

The `database-service` translates account-specific desired states into CloudNativePG. The projection code creates a `CNPG Cluster` utilizing a specified PostgreSQL image, together with resource allocations, storage configurations, bootstrap ownership, database setup, monitoring capabilities, and security parameters. Databases equipped with backup capabilities also obtain a Barman `Cloud ObjectStore` and `CNPG ScheduledBackup`. The reconciliation worker utilizes these resources, elucidates `CNPG` circumstances and instance status, and documents stable product conditions.

`CNPG` produces the application credential `Secret` within the customer namespace. The service dynamically interprets the customer connection string during its construction and does not retain the plaintext database password in the PostgreSQL platform. A constrained pool manager imposes overall and individual database connection restrictions to prevent simultaneous browser sessions from depleting ARM64 nodes or the administered cluster.

SQL execution permits a constrained script, secures a client-specific pool slot, initiates a transaction, and enforces `SET LOCAL statement_timeout`. Defaults impose a 15-second delay for the database, a 25-second overall deadline, a 256 KiB limit for scripts, and result truncation after 1,000 rows or 1 MiB. Every entry is encoded in JSON format throughout the collection process, ensuring that byte accounting aligns with the true response structure. SQL mistakes transform into organized execution outcomes, whereas connection or control-plane failures persist as service faults.

CSV and JSON imports are processed by streaming iterators and shown as persistent import tasks. Utilize and substitute with `pgx.CopyFrom`; upsert identifies the main key and generates conflict changes. A position is limited to 500,000 rows, and each mode utilizes a single transaction along with a server-side timeout. In replace mode, `TRUNCATE` and `COPY` intentionally operate within the same transaction; any parsing, constraint, timeout, or copy error results in a rollback of the truncation, safeguarding the current customer data.

6.5 Object Storage & Virtual Networking (`storage-service`)

`storage-service` presents logical buckets over a shared Garage S3 backend. Tenants never receive the Garage platform credential or submit a complete physical key. The service first proves ownership with the `(bucket_id, account_id)` lookup, sanitizes the requested key, and derives the backend prefix only from immutable identifiers. The shortened boundary below is drawn from the central prefix and service functions:

```
``go
```



```

func ForBucket(accountID, bucketID uuid.UUID) objectstore.Prefix {
    return objectstore.Prefix(
        fmt.Sprintf("acct/%s/%s", accountID, bucketID),
    )
}

func (s *Service) ownedTenantPrefix(ctx context.Context,
    accountID, bucketID uuid.UUID, requested string,
) (string, objectstore.Prefix, error) {
    if _, err := s.q.GetBucketByID(ctx, db.GetBucketByIDParams{
        ID: bucketID, AccountID: accountID,
    }); err != nil {
        return "", "", storage.MapError(err)
    }

    key, err := prefix.SanitizeKey(requested)

    if err != nil {
        return "", "", err
    }

    return key, prefix.ForBucket(accountID, bucketID), nil
}
...

```

The effective Garage key is `acct/<account_id>/<bucket_id>/<key>`. `SanitizeKey` rejects empty or oversized input, invalid UTF-8, control characters, absolute paths, literal `..` segments, slash and backslash variants, and selected percent-encoded traversal forms. Unicode NFC normalization precedes the final checks. Mutable bucket names are absent from physical addressing, so a rename cannot orphan stored data. All Garage operations accept the opaque `objectstore.Prefix` type, concentrating the isolation boundary in one auditable package.

A background usage collector claims stale buckets with `FOR UPDATE SKIP LOCKED`, scans Garage usage by prefix, and records time-series snapshots. Per-bucket deadlines prevent one slow listing from blocking

a cycle, failed claims are released for immediate retry, and retention pruning bounds platform storage. The latest snapshots support quota decisions without issuing an S3 scan on every request.

Virtual networks are constrained to RFC 1918 address space and validated against cluster ranges and prohibited account overlaps. Firewall rules use deterministic deny-overrides evaluation; representable allow sets become `networking.k8s.io/v1 NetworkPolicies`. Explicit deny rules with no corresponding allow and ICMP-only semantics cannot be faithfully expressed by Kubernetes NetworkPolicy, so reconciliation records partial or unenforced status rather than reporting false isolation. The VPC product is therefore a controlled namespace-policy abstraction, not a full overlay router.

6.6 Interactive WebSocket Terminal Gateway (`terminal-gateway`)

The terminal gateway isolates the powerful `pods/exec` permission from the public API gateway. Its handler orders the security-sensitive path as `redeem > revalidate > acquire slot > upgrade > attach > pump`. Ticket and compute checks occur before the HTTP upgrade, allowing meaningful 401, 429, or 503 responses rather than opaque WebSocket close failures.

Redemption uses Valkey `GETDEL`; a separate read and delete would permit concurrent reuse. Resource, address, and time bindings are checked after the atomic consume:

```
```go
raw, ok := cacheClient.GetDel(ctx, cache.NSTermTicket+ticketID)

if !ok {
 return Ticket{}, ErrTicketInvalid // absent, expired, or already consumed
}

var ticket Ticket

if err := json.Unmarshal(raw, &ticket); err != nil {
 return Ticket{}, ErrTicketInvalid
}

age := time.Since(time.Unix(ticket.IssuedAt, 0))

if ticket.InstanceID != instanceID ||
 (bindIP && ticket.ClientIP != clientIP) ||
 (maxAge > 0 && (age < 0 || age > maxAge)) {
 return Ticket{}, ErrTicketInvalid
}
```

```

}

return ticket, nil
...

```

The ticket only verifies if the gateway has recently validated an account. Consequently, `terminal-gateway` generates a service token specific to the audience and invokes the limited `exec-target` endpoint of `compute-service` for each connection. This RPC reassesses ownership, operational status, namespace, pod, and container. The gateway thereafter obtains a Valkey-supported account session slot and enhances the socket.

The `client-go/tools/remotecommand` module establishes an SPDY `Pods/exec` stream. Pipe-backed pumps link WebSocket frames to pod standard input and output. Binary control frames adjust the TTY via a non-blocking debounced queue, text frames retain unprocessed input, ping/pong ensures ongoing activity, and separate idle and maximum-lifetime deadlines provide warnings before to termination. A regional registry monitors compromised sockets since `net/http.Server.Shutdown` does not pause for them; draining declines new sessions, alerts active clients, and terminates outstanding connections before the deadline. The open and closure action is transmitted to IAM audit over a constrained best-effort route.

## 6.7 Identity, Access & Audit Subsystem (`iam-service`)

IAM possesses accounts, users, quotas, both managed and custom policies, API keys, provisioning status, and audit logs. The first OIDC account resolution process creates the account holder and establishes the default quota. Four managed roles are delineated as policies: The administrator grants unrestricted access; Editor oversees compute, database, storage, and network resources but is restricted from IAM modifications. A viewer gives read access, and an auditor enables resource and audit reads while prohibiting writes and deletions. Evaluation operates on a default-deny principle with deny overrides, so a corresponding denial cannot be counteracted by an additional allowance.

The generation of an API key yields plaintext a single time. IAM produces arbitrary data together with a lookup prefix, thereafter retaining just an Argon2id PHC-format string that includes a random 16-byte salt, a 32-byte derived value, and a memory cost of 64 MiB and employs constant-time comparison. This deliberately costly route is designated for automation credentials; regular browser traffic use OIDC.

User lifecycle procedures save local state and then use Authentik's administrative `/api/v3` endpoints for remote provisioning or status modifications. This resembles SCIM synchronization, although it does not conform to a specified SCIM endpoint. A periodic drift worker reconciles established Authentik groups, contrasts local and distant users, identifies absent or inconsistent records, and may utilize a corresponding remote identity. `pg_try_advisory_lock` guaranties that only a single IAM replica executes a pass, preventing drift detection from inadvertently overriding ambiguous local or remote statuses.

Audit events retain the actor, account, action, resource, outcome, metadata, request identifier, and timestamp. Mutation pathways are documented with an anticipated audit, and assessments dismiss an

unclassified novel mutation. PostgreSQL withdraws `UPDATE` and `DELETE` permissions on `iam.audit_log` from the application role, while preserving `INSERT` and `SELECT`. Append-only functionality is hence a privilege barrier inside the database rather than a standard for handlers.

## 6.8 Frontend Web Application (frontend)

The frontend is a React 19 SPA written in strict TypeScript 6 and built with Vite [4]. Feature directories contain API adapters, types, TanStack Query hooks, pages, and focused tests for each cloud product. Axios provides common transport behavior and translates the FCI error envelope into typed UI errors. OIDC access tokens are synchronized into the client, and protected routes initiate or require Authentik login.

TanStack Query manages server state, cache identifiers, mutation invalidation, retry strategies, and loading/error transitions. Zustand is limited to local browser state, including region selection, theme, notifications, modal status, and form preferences; cloud resources do not establish an alternative source of truth within the SPA. The terminal interface combines xterm.js with the console-ticket endpoint and an embedded WebSocket client. Vite divides frameworks, terminals, editors, and chart libraries into segments, with production endpoints sent via runtime `/config.js` instead of being included into the bundle.

Mock Service Worker provides an optional browser worker for development that is independent of the backend, along with a Node server for Vitest. Handlers represent models for account management, computation, databases, storage solutions, networking, identity and access management, and console interfaces. Production initiation omits mocks until specifically activated. React Testing Library, accessibility validations, stringent MSW request handlers, and communal query fixtures confirm both user-facing functionality and expectations for backend agreements.

## 7. Verification, Testing & Evaluation

The assessment concentrated on the accuracy of tenant segregation, resource lifecycle management, security measures, and the streamlined deployment framework. The initiative integrates automated unit and integration testing alongside API contract verification, infrastructure assessment, and relevant performance evaluations. Results reliant on hardware are differentiated from established acceptance criteria to prevent the misrepresentation of unperformed cluster tests as valid metrics.

### 7.1 Testing Strategy & Test Suites

**Unit tests:** The Go services employ table-driven testing methodologies for business logic, input validation, state transitions, token parsing, authorization protocols, S3 key sanitization, CIDR management, and terminal protocol frames. Tests executed using ``go test -race`` to ensure that concurrent workers, caches, and handlers are also scrutinized for data races. The frontend employs Vitest and React Testing Library to assess components, query hooks, routing, error conditions, and user interactions.

Integration assessments: Testcontainers [31]-driven suites initiate actual PostgreSQL [10] and Valkey [13] instances. PostgreSQL examinations utilize the integrated Goose [33] migrations and evaluate repository queries, constraints, transactions, advisory locks, and `FOR UPDATE SKIP LOCKED` work assertions. Valkey assessments confirm rate limitation, key expiration, revocation, session constraints, and atomic single-use terminal ticket redemption. Service-specific integration examinations also assess account segregation, rollback functionality, and alignment with a Kubernetes [5] test API as necessary.

API and contract validation: Handler tests confirm route arguments, authentication prerequisites, status codes, pagination, idempotency, and the standard problem-response envelope. The frontend employs Mock Service Worker to simulate identical computing, database, storage, networking, IAM, and terminal interfaces without necessitating an active backend. Rigorous unaddressed-request conduct aids in uncovering discrepancies between frontend anticipations and the described API.

Infrastructure assessments: The deployment repositories utilize `helm lint`, Helm template generation, `kubeconform` [35], and YAML verification prior to the acceptance of manifests. Fundamental smoke tests confirm container initialization, health endpoint functionality, Kubernetes [5] resource visualization, and architectural congruence. These verifications enhance, however, do not substitute, the conclusive deployment testing on the seven-node ARM64 cluster.

## 7.2 Requirements Traceability Matrix

The following matrix maps the Chapter 3 requirements to their principal components and verification evidence. “Passed” denotes focused automated tests in the repositories; “Verified” denotes implementation and declarative configuration confirmed by code and manifest analysis. “Partially verified” identifies a remaining hardware or high-availability acceptance gap.

ID	Requirement	Component(s)	Verification method	Status
FR-01	Authentication, account resolution, API keys, and RBAC	API Gateway, IAM, Authentik, platform-common	OIDC/JWT, Argon2id key, policy, role-boundary, and cross-account tests	Passed
FR-02	Compute-engine lifecycle, quota, backup, and cleanup	Compute Service, Kubernetes	CRUD/state-machine tests, PostgreSQL worker tests, Kubernetes projection and Kind lifecycle tests	Passed
NFR-01	Metrics, logs, traces, health checks, and dashboards	All services, Prometheus/Grafana/Loki/Tempo	Telemetry tests, health-route tests, ServiceMonitor and dashboard manifest validation	Verified

NFR-02	Reproducible, portable GitOps deployment	Ansible, Argo CD, Helm, Terraform, GitHub Actions	Playbook, chart, schema, workflow, sync-wave, and infrastructure validation	Verified
--------	------------------------------------------	---------------------------------------------------	-----------------------------------------------------------------------------	----------

## 8. Discussion, Limitations & Future Work

### 8.1 Post-Mortem & Objectives Realization

The initiative accomplished its primary goal: providing the fundamental interaction framework of a public cloud on a physical assembly of seven Raspberry Pi ARM64 devices. FCI offers compute engines tailored to specific accounts, managed PostgreSQL [10] databases, S3-compatible object storage, virtual networking, and firewall configurations, along with browser terminal access, identity and access management, quotas, and audit logs via a React [2] interface and a consolidated API gateway. Ansible [8], Argo CD [7], OpenBao [14], GitHub Actions, and the observability stack enhance outcomes beyond application code into a replicable platform-engineering framework.

The primary architectural inference is that container-native computing served as the effective abstraction for this hardware. Complete virtual machines would utilize an inordinate amount of memory, necessitate enhanced virtualization capabilities, and diminish workload density. Kubernetes [5] Deployments, PVCs, quotas, and NetworkPolicies offer a streamlined resource framework while maintaining a recognizable self-service cloud interface. The compromise is that workloads utilize the host kernel and lack the isolation or guest operating system adaptability provided by hardware virtualization.

The second significant result refers to asynchronous control-plane operations. PostgreSQL's row locking with `FOR UPDATE SKIP LOCKED` facilitated a resilient, multi-worker reconciliation queue without the need for an AMQP broker. The desired state, work accountability, attempts, and actual state closely align with the transactional system of record. This minimized memory consumption and operational intricacy on the Raspberry Pis, yet database polling would become increasingly inappropriate if the platform expanded to hyperscale event quantities.

### 8.2 Technical Limitations & Known Issues

**Hardware and architecture:** FCI presently functions as a singular K3s [6] cluster within a unified LAN. Replicated service pods can withstand some process or worker failures; nevertheless, there is an absence of a multi-region active-active control plane or a secondary disaster recovery site. A breakdown in power, network, ingress, or control-plane can therefore impact the entire platform.

**Storage and input/output:** The SD-card and USB storage of the Raspberry Pi exhibit constrained throughput and durability in comparison to server NVMe systems. Database imports, S3 activities, container image retrievals, and replicated-volume data transfers may contend for identical I/O and network resources. Storage replication mitigates some node-specific vulnerabilities; nonetheless, it does not substitute for off-cluster backups and verified recovery protocols.

**Networking:** Kubernetes [5] NetworkPolicy is unable to encapsulate all conventional firewall regulations. Explicit denial semantics and ICMP-specific regulations are only partially implementable using the conventional policy framework. FCI documents rules that are either unsupported or only partially enforced instead of asserting a more robust VPC guarantee than what the substrate can deliver.

**Terminal sessions:** Browser terminal sessions are transient. A gateway restart, target-pod reboot, network disruption, or maximum session timeout terminates the WebSocket and pods/exec stream. The disposable security ticket is not eligible for reapplication to retrieve the prior session, and the platform does not save terminal history.

**Database implementation:** The existing PostgreSQL deployment comprises a solitary CloudNativePG instance, resulting in a deficiency compared to the intended three-node high-availability configuration. Despite the utilization of several copies and a resilient reconciliation state by the stateless services, the failure of this database instance may disrupt IAM, audit, quota, and resource-management functions until restoration occurs.

### 8.3 Future Roadmap & Extensions

**Upcoming managed cloud services:** The frontend already identifies several product areas as “Coming Soon.” Managed Load Balancers will provide tenant-scoped Layer 4 and Layer 7 traffic distribution for customer workloads, including declarative listeners, backend membership, health checks, and TLS-aware ingress policies. Managed Kubernetes will expose Kubernetes-as-a-Service through tenant-isolated control planes and virtual clusters based on vCluster, avoiding the cost of a complete physical cluster for every account. Managed Event Streaming will introduce multi-tenant Apache Kafka topics, consumer groups, retention policies, and quota-controlled event pipelines. Managed Search and Analytics will provide Elasticsearch or OpenSearch clusters for distributed indexing, full-text search, and analytical workloads. Each service must follow the existing desired-state pattern, account ownership model, quota enforcement, audit classification, and GitOps-compatible operational controls before it is promoted from the frontend roadmap to a generally available capability.

**Database engine expansion:** database-service is planned to evolve from a PostgreSQL-specific controller into a common managed-database control plane. MySQL will provide a conventional client/server relational alternative, while SQLite will support lightweight, low-concurrency databases whose storage and lifecycle can be represented with persistent volumes and controlled service access. Engine-specific reconcilers, backup formats, credential handling, health models, connection limits, and import/export semantics must remain explicit; the existing CloudNativePG PostgreSQL implementation should not be weakened into a lowest-common-denominator abstraction.

**Hardware and community growth:** Incorporating x86-64 servers would enhance memory capacity, storage performance, and workload compatibility, while maintaining ARM64 nodes for optimal container operations. FCI is designed to be entirely open source and accessible for public contributions. All Patreon and community sponsorship revenue will be entirely allocated toward the reinvestment in physical servers, robust storage solutions, networking apparatus, and cluster capabilities. Enhancing publishing capacity increments, usage, and distribution regulations would render this sustainability framework clear and responsible.

**Dedicated compute profiles:** Dedicated compute profiles will provide hardware-level workload isolation and node affinity using Kata Containers, allowing tenants to execute security-sensitive workloads within lightweight virtualized container environments.

**Control-plane high availability:** The platform database will be enhanced to a three-instance CloudNativePG cluster including anti-affinity, automatic failover, and periodic backups to guarantee high availability.

## 9. Conclusion

This graduation project conceptualized, executed, launched, and assessed the Free Cloud Initiative as a self-sufficient cloud platform operating on a seven-node Raspberry Pi ARM64 cluster. The study tackled a pragmatic engineering inquiry: if the fundamental self-service functionality of a public cloud could be emulated on significantly resource-limited, community-operated hardware without depending on an extensive virtualization or private-cloud management system. The resultant solution illustrates that a meticulously constrained, container-native framework can deliver a reliable cloud experience while being comprehensible, replicable, and financially attainable.

The primary software offering is a cohesive collection of multi-tenant cloud components. FCI offers container-native computing engines; PostgreSQL [10] databases managed by CloudNativePG [11], S3 object storage that is tenant-isolated and supported by Garage [12], virtual networks with firewall regulations aligned to Kubernetes [5] `NetworkPolicy`; and a terminal bridge from WebSocket to  `pods/exec`. IAM provides administration of account lifecycles, API keys, default-deny and deny-overrides role-based access control, quotas, and database-enforced immutable audit logs. A React [2] 19 and TypeScript [3] single-page application offers these functionalities via a cohesive user interface, while the Go [1] API gateway converts external OIDC or API-key credentials into ephemeral, audience-specific internal tokens.

The contribution of platform engineering is equally important. Ansible [8] configures the physical nodes and initializes K3s [6] along with the foundational security measures. Argo CD [7] utilizes the App-of-Apps paradigm and sequential synchronization waves to declaratively align the cluster. OpenBao [14] and External Secrets [17] provide application credentials without including them into workload manifests. Reusable GitHub Actions processes evaluate, inspect, construct, and disseminate multi-architecture pictures, while Prometheus [22], Grafana [23], Loki [25], Tempo [26], Alloy, and OpenTelemetry [24] provide metrics, dashboards, logs, and traces. Collectively, these approaches render the deployment and its operational condition replicable instead of regarding infrastructure as an unrecorded need.

FCI therefore provides more than only a suite of services. It illustrates that contemporary public-cloud interaction frameworks and resilient account-specific architectures may be constructed affordably using open-source elements on basic hardware. The platform's public repositories, educational observability objectives, and sponsorship framework create a foundation for community engagement: it is free to utilize, and all monies from Patreon and community sponsorship are designated to enhance physical server and storage capabilities. Through transparent administration and ongoing validation, the initiative



may evolve from an academic prototype into a sustainable open-source cloud environment and platform-engineering laboratory.

## 10. References

- [1] <https://go.dev/doc/>
- [2] <https://react.dev/reference/react>
- [3] <https://www.typescriptlang.org/docs/>
- [4] <https://vite.dev/guide/>
- [5] <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [6] <https://docs.k3s.io/>
- [7] <https://argo-cd.readthedocs.io/en/stable/>
- [8] <https://docs.ansible.com/>
- [9] <https://developer.hashicorp.com/terraform/docs>
- [10] <https://www.postgresql.org/docs/16/>
- [11] <https://cloudnative-pg.io/documentation/1.24/architecture/>
- [12] <https://garagehq.deuxfleurs.fr/documentation/design/architecture/>
- [13] <https://valkey.io/topics/>
- [14] <https://openbao.org/docs/concepts/auth/>
- [15] <https://docs.goauthentik.io/>
- [16] <https://cert-manager.io/docs/>
- [17] <https://external-secrets.io/latest/>
- [18] <https://kyverno.io/docs/>
- [19] <https://longhorn.io/docs/1.7.2/architecture/core-concepts/>
- [20] <https://doc.traefik.io/traefik/routing/providers/kubernetes-ingress/>
- [21] <https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/>
- [22] <https://prometheus.io/docs/introduction/overview/>
- [23] <https://grafana.com/docs/grafana/latest/>

- [24] <https://opentelemetry.io/docs/>
- [25] <https://grafana.com/docs/loki/latest/>
- [26] <https://grafana.com/docs/tempo/latest/>
- [27] <https://tanstack.com/query/latest/docs/framework/react/overview>
- [28] <https://zustand.docs.pmnd.rs/>
- [29] <https://xtermjs.org/docs/>
- [30] <https://mswjs.io/docs/>
- [31] <https://testcontainers.com/>
- [32] <https://docs.sqlc.dev/en/latest/>
- [33] <https://pressly.github.io/goose/>
- [34] <https://golangci-lint.run/docs/>
- [35] <https://github.com/yannh/kubeconform>
- [36] <https://www.vcluster.com/docs/>
- [37] <https://kafka.apache.org/documentation/>
- [38] <https://www.elastic.co/docs>
- [39] <https://docs.opensearch.org/latest/>
- [40] <https://dev.mysql.com/doc/>
- [41] <https://www.sqlite.org/docs.html>
- [42] <https://kubevirt.io/user-guide/>
- [43] <https://katacontainers.io/>